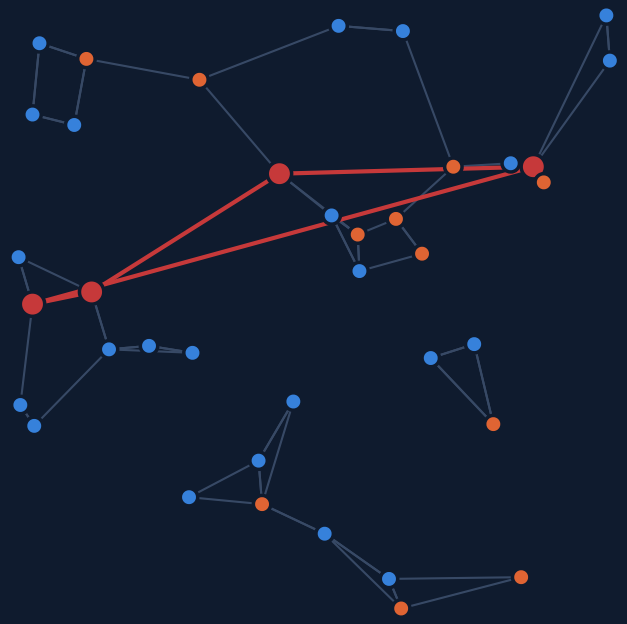
FraudLens

Solution guide · October 2026



Real-time fraud detection for banking-correspondent (CSP) transactions

# Catch the fraud while the transaction is still happening.

How the solution works, and a dedicated page for each of the 22 fraud patterns it captures: what the fraud looks like, exactly how it is detected, and what it found in a 1.6-lakh-transaction test run.

| 22 | ~10 µs | 0.3 ms | 3.9 s |
|---|---|---|---|
| fraud patterns (14 yours + 8 added) | to score one transaction | webhook reply incl. network | to upload and analyse 1,63,569 rows |

# Contents


**1 · At a glance** 3

**2 · Architecture** 4

**3 · Transaction scoring** 5

**4 · CSP scoring** 6

**5 · Dashboard tour** 7

**6 · Dashboard tour** 8

Pattern 01 · Split transactions 9

Pattern 02 · Round trip (money loops) 10

Pattern 03 · 1% extra charge (overcharging) 11

Pattern 04 · ₹1 test deposit, then cash-out 12

Pattern 05 · Fund transfer to relatives / own accounts 13

Pattern 06 · ₹436 transactions (PMJJBY premium) 14

Pattern 07 · Sub-KO usage 15

Pattern 08 · ₹50 transactions 16

Pattern 09 · Attendance 17

Pattern 10 · Number of transactions 18

Pattern 11 · Spike / drop 19

Pattern 12 · Night transactions 20

Pattern 13 · Anomaly vs peers 21

Pattern 14 · Network / fraud rings 22

Pattern 15 · High-value night transaction 23

Pattern 16 · High velocity 24

Pattern 17 · Pass-through / mule account 25

Pattern 18 · Duplicate transaction 26

Pattern 19 · Threshold avoidance (structuring) 27

Pattern 20 · Amount outlier 28

Pattern 21 · Dormant account reactivated 29

Pattern 22 · Success after failed attempts 30

**Appendix A · Input and API** 31

**Appendix B · Configuration** 32

**Appendix C · Testing and next steps** 33

1

# The solution at a glance

FraudLens is a single program that keeps every transaction in memory, checks each new one against 18 real-time rules the moment it arrives, and continuously builds CSP-level analytics on top of the same data.

### ① PUT TRANSACTIONS IN

**Upload a sheet**: CSV, Excel (.xlsx) or JSON, from the dashboard or the API. Column names are matched loosely ("Debit A/c", "Agent ID", "Txn Date"...).

**Or call the webhook** for each transaction as it happens. Retries with the same transaction id are safe.

### ② GET AN INSTANT ANSWER

Every transaction gets a **risk score (0–1)**, an action — ALLOW REVIEW BLOCK — and the reasons, in microseconds.

Suspicious ones become **alerts** that stream live to the dashboard.

### ③ INVESTIGATE ON THE DASHBOARD

**CSP risk table** with every CSP-level rule, a **network view** of fraud rings, transaction look-up, and CSV export for reporting.

Data survives restarts (a log on disk rebuilds the cache).

## The 22 patterns

| | | | | | |
|---|---|---|---|---|---|
| 01 | Split transactions | YOURS | 02 | Round trip (money loops) | YOURS |
| 03 | 1% extra charge (overcharging) | YOURS | 04 | ₹1 test deposit, then cash-out | YOURS |
| 05 | Fund transfer to relatives / own accounts | YOURS | 06 | ₹436 transactions (PMJJBY premium) | YOURS |
| 07 | Sub-KO usage | YOURS | 08 | ₹50 transactions | YOURS |
| 09 | Attendance | YOURS | 10 | Number of transactions | YOURS |
| 11 | Spike / drop | YOURS | 12 | Night transactions | YOURS |
| 13 | Anomaly vs peers | YOURS | 14 | Network / fraud rings | YOURS |
| 15 | High-value night transaction | ADDED | 16 | High velocity | ADDED |
| 17 | Pass-through / mule account | ADDED | 18 | Duplicate transaction | ADDED |
| 19 | Threshold avoidance (structuring) | ADDED | 20 | Amount outlier | ADDED |
| 21 | Dormant account reactivated | ADDED | 22 | Success after failed attempts | ADDED |

## Test run on synthetic data

**1,63,569**
transactions · 80 CSPs · 60 days

**1,508**
alerts (33 high)

**6 / 41**
to block / to review

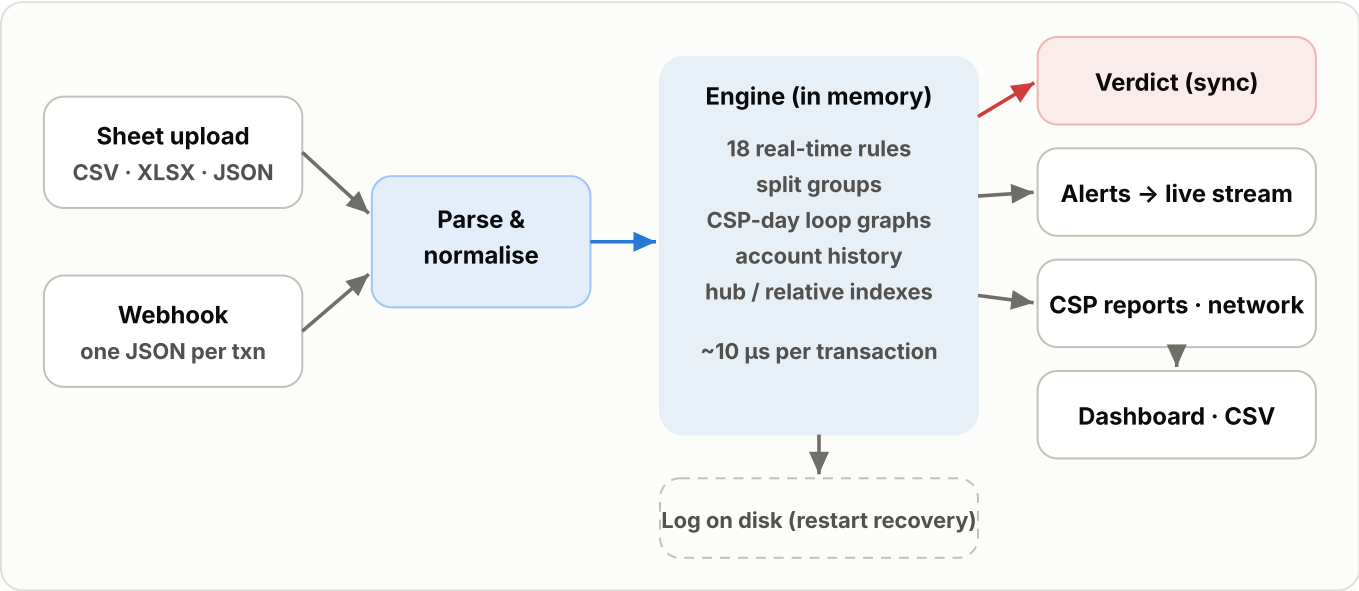
**18 / 19**
planted fraud CSPs ranked in the top 25 by risk

The test data comes from the included generator (`cmd/gen`): realistic background traffic plus every pattern planted at a known CSP. Real data will need threshold tuning.

2

# Architecture and data flow

One program, one in-memory engine. Both inputs go through the same parser and the same rules, so a sheet and a live webhook give identical results.




### What happens to each transaction

1. **Parse.** Columns are mapped loosely; amounts like "Rs. 5,050" or "₹4,950.00" and dates like 27/10/2026 14:05 or Excel serial dates are understood. Uploaded rows are **sorted by time** first.
2. **Normalise.** Type → deposit / withdrawal / transfer / loan / initial deposit; channel → ONUS/OFFUS; status → success/failed; masked accounts recognised.
3. **Evaluate.** All real-time rules run against the cached state (each does a constant amount of work, so cost does not grow with history).
4. **Store & answer.** The transaction joins the cache and the log; the verdict goes back to the caller; alerts go to the live feed.
5. **Analyse.** CSP reports and the network are recomputed on demand from the cache and cached until new data arrives.

### Why Go

- **Fast:** compiled, with cheap in-memory maps. About 100,000 transactions per second on one core.
- **Simple to maintain:** standard library only, no third-party packages. Each rule is a short, readable function.
- **Easy to run:** one file with the dashboard built in. Runs on a laptop, a VM or in Docker.

### Code map

| | |
|---|---|
| `internal/engine/realtime.go` | the real-time rules |
| `internal/engine/reports.go` | CSP analytics and scoring |
| `internal/engine/network.go` | cycles, hubs, rings |
| `internal/ingest/` | CSV / XLSX / JSON readers |
| `internal/server/` | API, live stream, dashboard |
| `cmd/gen` | synthetic test-data generator |

3

# How a transaction is scored

Each rule that fires adds a **signal** with a weight between 0 and 1. Signals are combined so that several medium signals add up, but the score never exceeds 1.

$$risk = 1 - (1 - w_1) \times (1 - w_2) \times \ldots$$

| ALLOW | REVIEW | BLOCK |
|---|---|---|
| below 0.5 | 0.5 – 0.8 | 0.8 and above |

### Worked example (real webhook call)

A ₹5,050 withdrawal from account 301546469923 at CSP0013, which had received a ₹1 test deposit two days earlier:

| | | |
|---|---|---|
| 1% extra charge | 0.4 | keep 0.6 |
| ₹1 test deposit then cash-out | 0.8 | keep 0.2 |
| **Risk** | **0.88** | 1 − 0.6 × 0.2 → **BLOCK** |

Engine time: ~40 µs on this cold call, ~5–10 µs typical. Thresholds (0.5 / 0.8) and every weight are configurable.

### Signal weights

| SIGNAL | SEVERITY | WEIGHT |
|---|---|---|
| ₹1 test deposit then cash-out | High | 0.8 |
| Round trip | High | 0.7 |
| Pass-through / mule | High | 0.65 |
| Success after failed attempts | High | 0.55 |
| Split · Hub account · Velocity · Threshold avoidance · Dormant | Medium | 0.45 |
| 1% extra charge · Relative transfer · Duplicate · Amount outlier | Medium | 0.4 |
| High-value night | Medium | 0.35 |
| ₹436 (PMJJBY) | Low | 0.3 |
| Night (tag) | Info | 0.1 |
| Sub-KO · ₹50 (tags) | Info | 0.05 |

Tags add to the score and appear in the reply, but are not stored as alerts on their own.

### The same call in the dashboard (Upload & API → Check one transaction now)



4

# How a CSP is scored

Each CSP gets a 0–100 risk score from the same combining formula, applied to CSP-level flags. Every flag is shown in plain words next to the CSP, so the score is always explainable.

| CSP FLAG | WEIGHT | WHEN |
|---|---|---|
| ₹1 test deposit + cash-out | 0.7 | any test-deposit account |
| Round trip Group 1 / 2 / 3 / other | 0.6 / 0.45 / 0.3 / 0.15 | loops > 0 |
| Repeated 1% extra charges / 1% extra charges | 0.55 / 0.25 | ≥ 3 and ≥ 1% / ≥ 0.5% of txns |
| Peer anomaly high / medium | 0.5 / 0.3 | score ≥ 0.8 / ≥ 0.6 |
| Transfers to relative / own accounts | 0.45 | any qualifying destination |
| High split share / Split transactions | 0.4 / 0.2 | ≥ 10% / ≥ 5% of txns |
| Night transactions above 20% | 0.4 | night share > 20% |
| Part of a suspicious network | 0.4 | member of a ring |
| Pass-through / mule | 0.35 | any alert |
| Repeated ₹436 debits | 0.3 | ≥ 3 |
| Threshold avoidance | 0.3 | any alert |
| Spike/drop red / amber | 0.3 / 0.15 | > 20% / ≥ 15% |
| Success after failed attempts | 0.25 | any alert |
| High sub-KO share · Low attendance · Unusual count · Many ₹50 · Velocity | 0.2 | see each pattern |
| Uses hub accounts · Duplicate · Dormant · High-value night | 0.15 | any |

| Low | Medium | High |
|---|---|---|
| 1 – 49 | 50 – 79 | 80 – 100 |

### Top 13 CSPs in the test run

| | | |
|---|---|---|
| CSP0007 | 88 | Round trip Group 1, Peer anomaly high, Part of a suspicious network |
| CSP0073 | 85 | Transfers to relative/own accounts, Part of a suspicious network, Pass-through / mule |
| CSP0013 | 82 | ₹1 test deposit + cash-out, Part of a suspicious network |
| CSP0003 | 72 | Split transactions, Spike/drop red, Peer anomaly high |
| CSP0005 | 72 | Transfers to relative/own accounts, Part of a suspicious network, Uses hub accounts |
| CSP0047 | 66.9 | Spike/drop amber, Part of a suspicious network, Pass-through / mule |
| CSP0037 | 66 | Unusual transaction count (high), Peer anomaly high, Uses hub accounts |
| CSP0061 | 61 | Part of a suspicious network, Pass-through / mule |
| CSP0041 | 60.8 | Unusual transaction count (high), Spike/drop red, Peer anomaly medium |
| CSP0031 | 60 | Low attendance, Peer anomaly high |
| CSP0023 | 59.2 | High sub-KO share, Part of a suspicious network, Uses hub accounts |
| CSP0032 | 56.7 | Spike/drop amber, Part of a suspicious network, Uses hub accounts |
| CSP0011 | 55 | Repeated 1% extra charges |

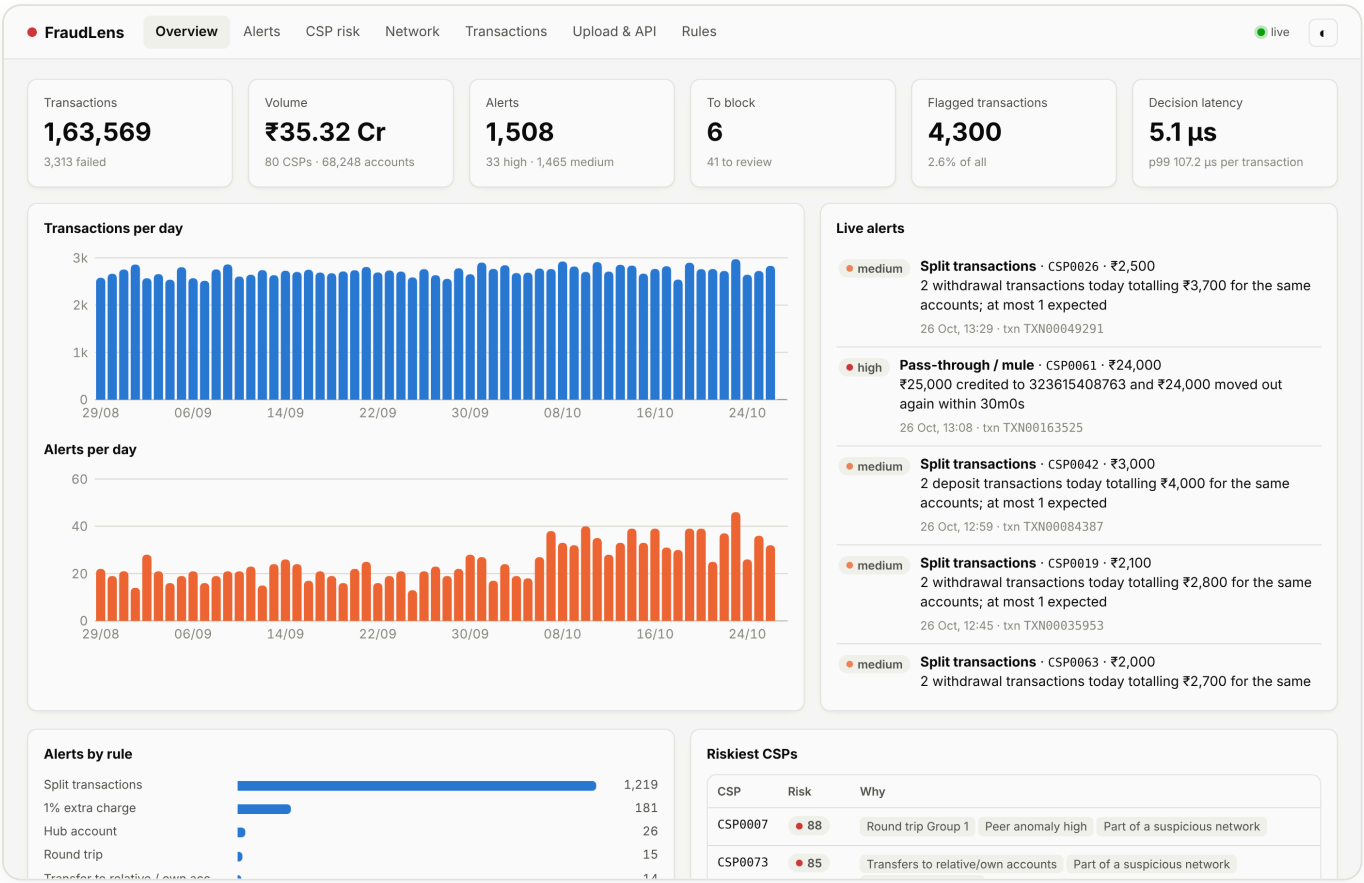
11 of these 13 are planted fraud CSPs. CSP0005 and CSP0032 are here because they paid the planted hub account and sit in the same ring. 18 of 19 planted CSPs rank in the top 25; the miss is CSP0067 (duplicate postings only, a low-weight CSP flag), at 43rd.

5

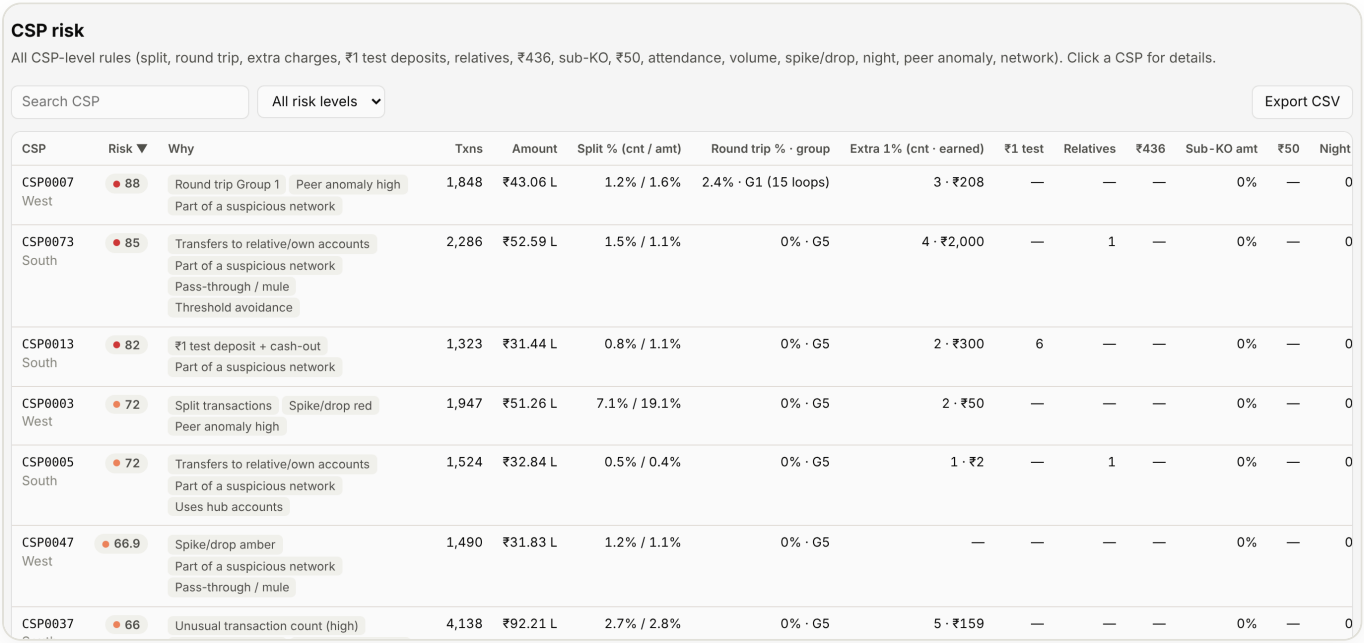
# Dashboard tour

Open `http://localhost:8080`. Everything updates live as transactions arrive.

### Overview: totals, daily volume and alerts, live alert feed, alerts by rule, riskiest CSPs



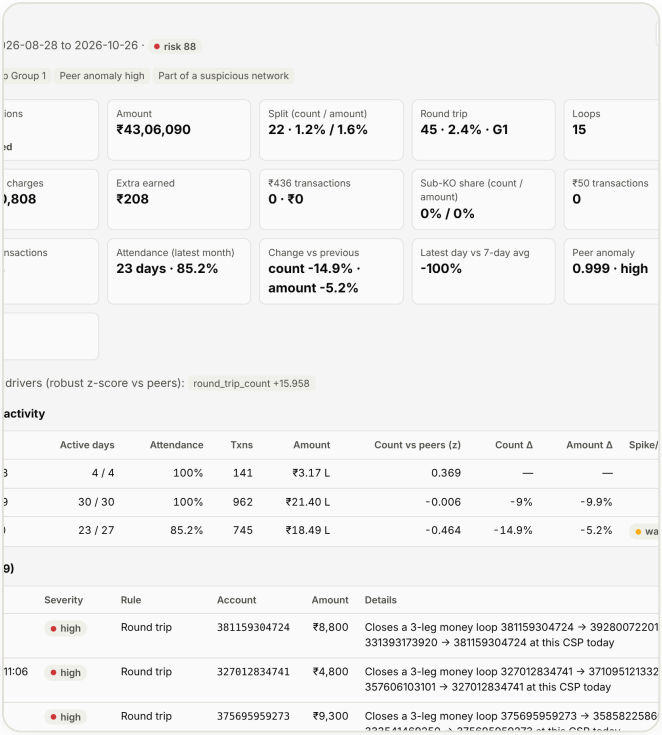

### CSP risk: one row per CSP with every CSP-level rule; sortable, searchable, exportable



6

# Dashboard tour (continued)

## CSP detail: every number, monthly activity, alerts



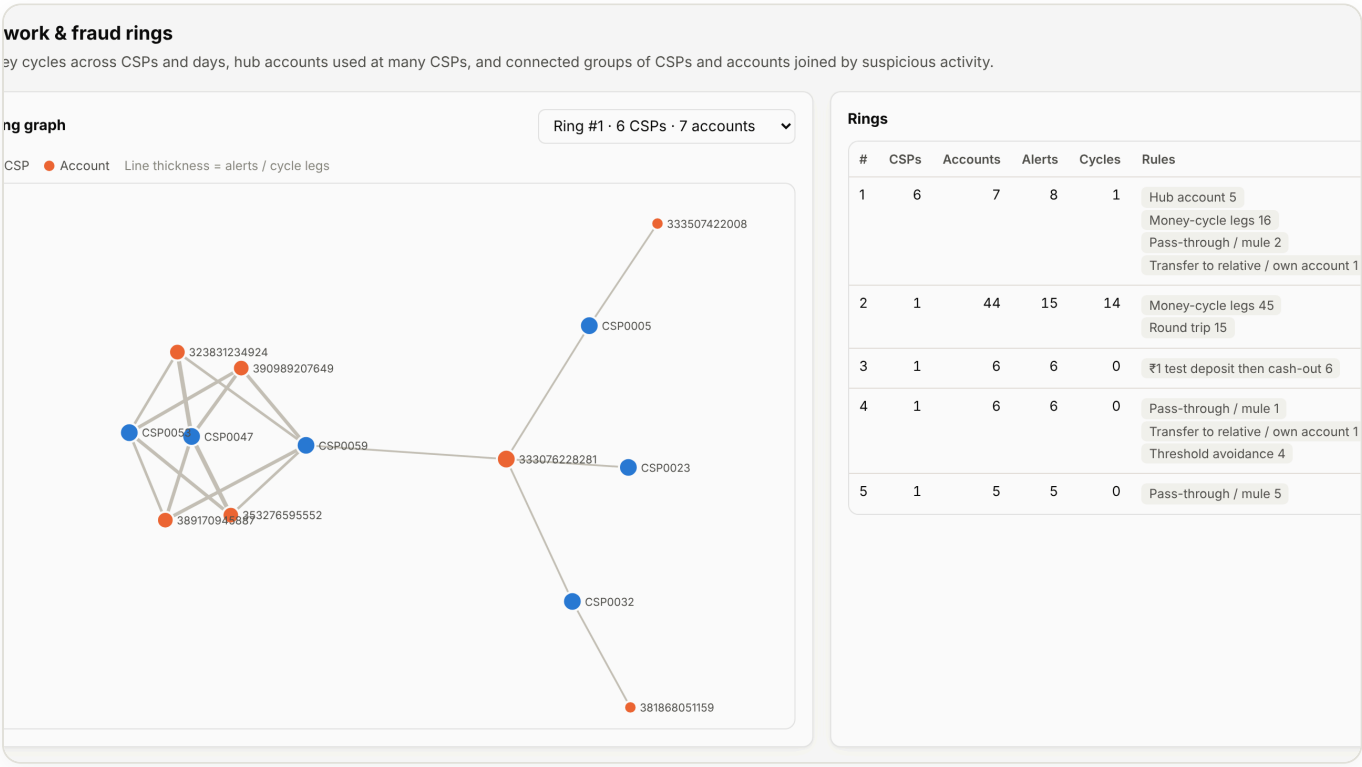

## Other pages

- **Alerts**: every alert, filter by rule / severity / CSP / account, CSV export.
- **Transactions**: look up any transaction id and see why it scored what it did.
- **Upload & API**: drag-and-drop a sheet; test a single transaction against the webhook.
- **Rules**: the live rule catalogue and the active thresholds.

### INVESTIGATION FLOW

1. Spot a CSP in **CSP risk** or a high alert in the live feed.
2. Open the CSP: read its flags, monthly table and alerts.
3. Click an alert to see the transaction and the related transactions (e.g. the other legs of a loop).
4. Check **Network** to see who else is connected.
5. Export alerts or the CSP table for the case file.

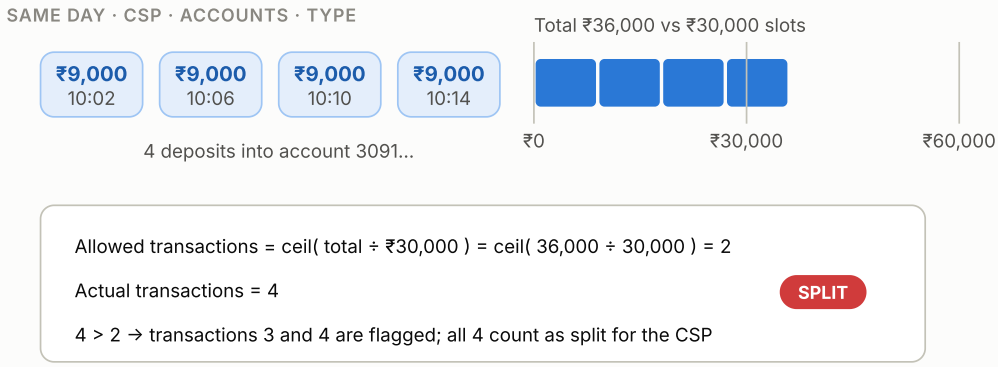
## Network: fraud rings, cross-CSP money cycles and hub accounts



01 YOUR RULE 1 Real-time · every transaction CSP analytics Medium weight 0.45

# Split transactions

One large transaction broken into several smaller ones, so each piece stays under a limit, attracts less attention, or earns the CSP more commission.




## The fraud in practice

A customer comes to deposit ₹36,000. The CSP posts it as four deposits of ₹9,000 to the same account within a few minutes. Each piece looks harmless, but together they could have been just two transactions under the ₹30,000 unit. Splitting inflates commission and hides the real amount.

## How FraudLens detects it

1. Group transactions by **day + CSP + from account + to account + type**.
2. Treat a **Money Transfer with zero customer charge as a Deposit** (it joins the deposit group).
3. Keep a running count and total per group. Allowed = **ceil(total ÷ ₹30,000)**.
4. When count > allowed, the new transaction is flagged and the whole group counts as split.
5. Per CSP: **split count %** and **split amount %** of all its transactions.

## Worked example

| | | | | |
|---|---|---|---|---|
| 10:02 | Deposit | → 3091… | ₹9,000 | |
| 10:06 | Deposit | → 3091… | ₹9,000 | |
| 10:10 | Deposit | → 3091… | ₹9,000 | SPLIT |
| 10:14 | Deposit | → 3091… | ₹9,000 | SPLIT |

Third and fourth deposits get the **split** signal (weight 0.45).

## Thresholds

| | |
|---|---|
| `split_unit` | **₹30,000** |
| `split_share_pct` | **5% (CSP flag; 10% = high)** |

### WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)


```
{"rule": "split", "severity": "medium", "csp_id": "CSP0026", "txn_id": "TXN00049291", "amount": 2500,
"message": "2 withdrawal transactions today totalling ₹3,700 for the same accounts; at most 1 expected",
"account": "367289191497", "related_txns": ["TXN00049287", "TXN00049291"]}
```


### WHERE YOU SEE IT

- Verdict signal + alert `split`
- CSP columns: Split % (count / amount)
- CSP flag: "Split transactions" (+0.2) or "High split share" (+0.4)

### IN THE TEST RUN

1,219 split alerts overall. Planted CSP0003: **138 split transactions, 7.1% of count and 19.1% of amount**, against a median CSP of 1.3%. Risk score 72.

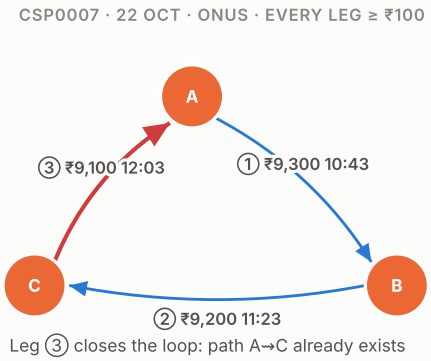
**When it fires.** Ask the CSP for the customer's original request; check whether the same customers appear in other split groups; recover the excess commission.

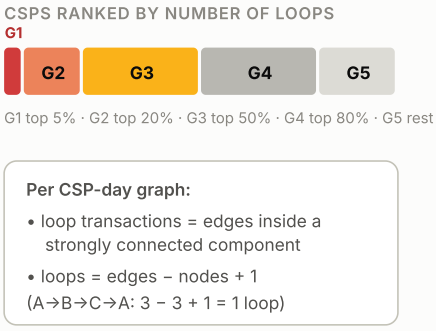
**Watch-outs.** Real customers sometimes deposit twice a day, so the median CSP shows ~1% split. That is why the CSP flag only fires at ≥ 5%.

02 YOUR RULE 2 Real-time · every transaction CSP analytics Network High weight 0.7

# Round trip (money loops)

Money sent around a circle of accounts and back to where it started on the same day, at the same CSP, to manufacture transactions and commission.




## The fraud in practice

The CSP moves ₹9,300 from A to B, B sends ₹9,200 to C, and C sends ₹9,100 back to A. No real money changed hands, but three commissionable transfers were created. The same trick inflates volume targets.

## How FraudLens detects it

1. Build a directed account graph **per CSP per day** from transfers that are ONUS, ≥ ₹100, unmasked (no XXXXXX) and not OFFUS.
2. When a new transfer u → v arrives, search (breadth-first, up to 8 hops) for an existing path v ⇝ u. If found, this transfer **closes a loop**.
3. For CSP reporting: transactions inside a strongly connected component are round-trip transactions; **loops = edges − nodes + 1** per component.
4. Rank CSPs by loops: **top 5% Group 1, top 20% Group 2, top 50% Group 3, top 80% Group 4, rest Group 5**.

## Worked example

| | | | | |
|---|---|---|---|---|
| 10:43 | Transfer | A → B | ₹9,300 | |
| 11:23 | Transfer | B → C | ₹9,200 | |
| 12:03 | Transfer | C → A | ₹9,100 | LOOP |

The third leg gets **round_trip** (weight 0.7, high). All three legs count as round-trip transactions.

## Thresholds

| | |
|---|---|
| `round_trip_min_amount` | **₹100** |
| `round_trip_max_depth` | **8 hops** |
| `Loop size` | **≥ 2 transactions** |

### WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)


```
{"rule": "round_trip", "severity": "high", "csp_id": "CSP0007", "txn_id": "TXN00163331", "amount": 8800,
"message": "Closes a 3-leg money loop 381159304724 → 392800722014 → 331393173920 → 381159304724 at this CSP
today", "account": "381159304724", "related_txns": ["TXN00163329", "TXN00163330", "TXN00163331"]}
```


### WHERE YOU SEE IT

- Verdict signal + alert `round_trip` with the loop path
- CSP columns: Round trip % (count / amount), loops, group
- CSP flag: Group 1 +0.6 · Group 2 +0.45 · Group 3 +0.3

### IN THE TEST RUN

15 loop alerts, all at planted CSP0007: **45 transactions on loops, 15 loops, 2.4% of count / 8.1% of amount → Group 1**. Every other CSP: Group 5. Risk 88 (highest).

**When it fires.** Hold commission on the loop transactions; call the account holders on the loop to confirm they made the transfers; check whether the accounts belong to the CSP or family.

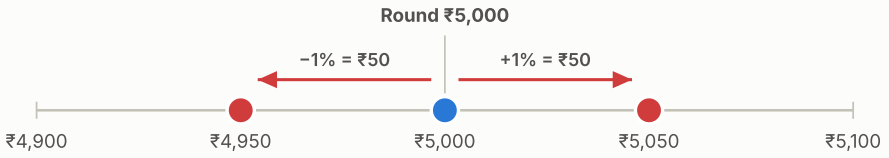
**Watch-outs.** Loops are found per CSP and day. A loop spread across CSPs or days is caught by the network view (pattern 14) instead.

03 YOUR RULE 3 Real-time · every transaction CSP analytics Medium weight 0.4

# 1% extra charge (overcharging)

The CSP quietly keeps 1% of the customer's money by posting a slightly different amount from what the customer asked for.

ROUND FIGURE: NEAREST ₹100 BELOW ₹1,000 · NEAREST ₹1,000 FROM ₹1,000 UP




**Withdrawal / transfer**
amount = rounded-down + 1%
₹5,050 = ₹5,000 + ₹50 → flag

**Deposit**
amount = rounded-up − 1%
₹4,950 = ₹5,000 − ₹50 → flag

## The fraud in practice

A customer asks to withdraw ₹5,000. The CSP debits ₹5,050 from the account, hands over ₹5,000 and pockets ₹50. For deposits it works the other way: the customer hands over ₹5,000 and only ₹4,950 is credited.

## How FraudLens detects it

1. Round figure: nearest **₹100** for amounts below ₹1,000, nearest **₹1,000** from ₹1,000 up.
2. **Withdrawal / transfer:** flag if amount = rounded-down figure + 1% (₹5,050 on ₹5,000, ₹1,010 on ₹1,000, ₹909 on ₹900).
3. **Deposit:** flag if amount = rounded-up figure − 1% (₹4,950 on ₹5,000, ₹990 on ₹1,000).
4. Amounts are held as integer paise, so the comparison is exact.
5. Per CSP: extra transaction count, extra transaction amount and **extra amount earned**.

## Worked example

| | | | | |
|---|---|---|---|---|
| 09:15 | Withdrawal | 3021… | ₹5,050 | +₹50 |
| 10:40 | Deposit | 3388… | ₹4,950 | −₹50 |
| 11:05 | Withdrawal | 3102… | ₹5,000 | |

Rows 1 and 2 get **extra_charge** (weight 0.4). Row 3 is a clean round amount.

## Thresholds

| | |
|---|---|
| `extra_charge_min_base` | **₹100** |
| `extra_charge_share_pct` | **0.5% (CSP flag; 1% = repeated)** |
| `CSP minimum` | **≥ 3 extra transactions** |

**WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)**


```
{"rule": "extra_charge", "severity": "medium", "csp_id": "CSP0016", "txn_id": "TXN00030180", "amount": 10100,
"message": "withdrawal/transfer is ₹10,100 on a round ₹10,000: likely ₹100 extra taken from the customer",
"account": "362901468280"}
```


### WHERE YOU SEE IT

- Verdict signal + alert `extra_charge`
- CSP columns: Extra 1% (count · earned)
- CSP flag: "1% extra charges" (+0.25) / "Repeated 1% extra charges" (+0.55)

### IN THE TEST RUN

181 matches overall, mostly natural coincidences (median CSP: 2). Planted CSP0011: **31 extra transactions = 1.5% of its volume, ₹1,706 earned** → "Repeated".

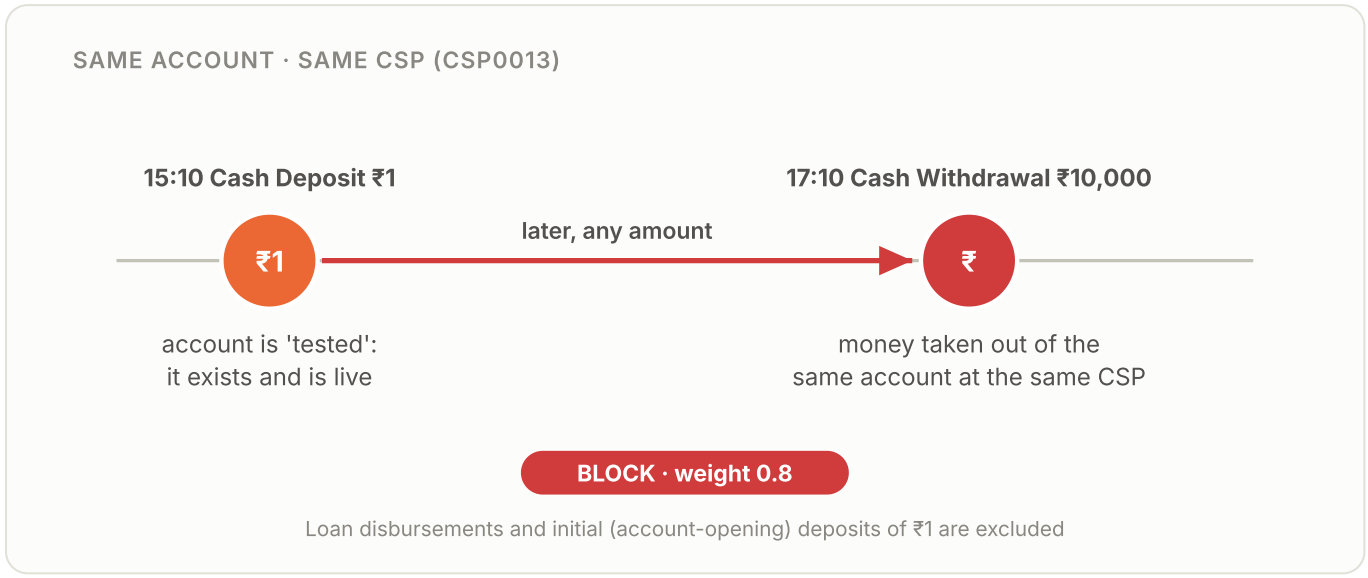
**When it fires.** Call a sample of the affected customers to confirm what they actually received or deposited; refund the difference; warn or suspend the CSP if it repeats.

**Watch-outs.** Some customers genuinely withdraw ₹1,010 or ₹2,020. A single match is a weak signal; the CSP-level share is what matters.

04 YOUR RULE 4 Real-time · every transaction CSP analytics High weight 0.8

# ₹1 test deposit, then cash-out

A tiny ₹1 deposit used to check that an account is live and controllable, followed by money being taken out of it.




## The fraud in practice

Fraudsters who have obtained someone's account details first send ₹1 to confirm the account works. Once confirmed, they withdraw or transfer the balance out through the same CSP, often one who is complicit.

## How FraudLens detects it

1. Record every **₹1 deposit** per (CSP, account). Loan disbursements and initial (account-opening) deposits are excluded.
2. When a **withdrawal or transfer out** of that account happens later **at the same CSP**, flag it.
3. Out-of-order input is handled: if the cash-out is already stored when the ₹1 deposit arrives, the deposit is flagged instead.
4. Per CSP: the list of test-deposit accounts.

## Worked example

| | | | | |
|---|---|---|---|---|
| 24 Oct 15:10 | Deposit | → 3015… | ₹1 | TEST |
| 24 Oct 17:10 | Withdrawal | 3015… → | ₹10,000 | FLAG |

The withdrawal gets **test_deposit_cashout** (weight 0.8). On its own that is a **BLOCK**.

## Thresholds

| | |
|---|---|
| `test_deposit_amount` | **₹1** |
| `Time limit` | **none: any later cash-out** |
| `Scope` | **same CSP + same account** |

### WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)


```
{"rule": "test_deposit_cashout", "severity": "high", "csp_id": "CSP0013", "txn_id": "TXN00163407", "amount": 10000, "message": "Cash-out of ₹10,000 from 301546469923, which received a ₹1 test deposit at this CSP on 24 Oct 15:10", "account": "301546469923", "related_txns": ["TXN00163406", "TXN00163407"]}
```


### WHERE YOU SEE IT

- Verdict signal + alert `test_deposit_cashout`
- CSP column: ₹1 test (accounts)
- CSP flag: "₹1 test deposit + cash-out" (+0.7)

### IN THE TEST RUN

Planted at CSP0013: **6 of 6 cases caught**, 6 accounts listed. CSP0013 risk 82 (high).

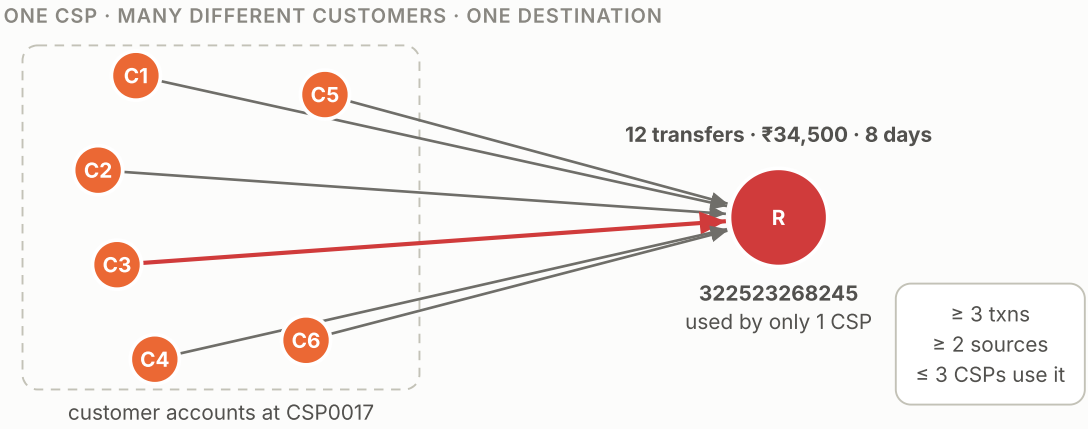
**When it fires.** Hold the cash-out if it is still pending; contact the real account holder; find who made the ₹1 deposit; review the CSP for other test accounts.

**Watch-outs.** Masked account numbers are skipped, because different customers would collapse into one key.

05 YOUR RULE 5 Real-time · every transaction CSP analytics Medium weight 0.4

# Fund transfer to relatives / own accounts

A CSP repeatedly routing customers' money into one account that almost nobody else uses: typically the CSP's own or a relative's.




## The fraud in practice

Customers ask the CSP to make transfers or withdrawals. Instead, the CSP sends the money to an account they control. Over days, many different customers' money ends up in the same destination, which no other CSP ever pays into.

## How FraudLens detects it

1. Consider only **successful ONUS withdrawals / transfers** with a destination account.
2. Track, for each destination, **how many CSPs** send to it, and for each (CSP, destination): transaction count, amount, days used and distinct source accounts.
3. Flag when the destination is used by **≤ 3 CSPs**, this CSP has used it **≥ 3 times**, from **≥ 2 different customer accounts**.
4. Per CSP: each such account with times used, transaction count and amount.

## Worked example

| | | | | |
|---|---|---|---|---|
| Day 1 | Transfer | C1 → R | ₹1,500 | |
| Day 2 | Transfer | C2 → R | ₹1,750 | |
| Day 4 | Transfer | C3 → R | ₹2,000 | FLAG |

Third transfer into R gets **relative_transfer** (weight 0.4), as does every later one.

## Thresholds

| | |
|---|---|
| relative_max_csps | **3** |
| relative_min_txns | **3** |
| relative_min_sources | **2 (our addition)** |

### WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)


```
{"rule": "relative_transfer", "severity": "medium", "csp_id": "CSP0017", "txn_id": "TXN00163411", "amount": 1750, "message": "This CSP has sent 12 transactions (₹34,500) to 322523268245 from 12 account(s); destination used by only 1 CSP(s)", "account": "322523268245"}
```


### WHERE YOU SEE IT

- Verdict signal + alert `relative_transfer`
- CSP column: Relatives (count) + table in CSP detail
- CSP flag: "Transfers to relative/own accounts" (+0.45)

### IN THE TEST RUN

Planted at CSP0017: account 322523268245 received **12 transfers (₹34,500) from 12 different customers over 8 days**; used by no other CSP.

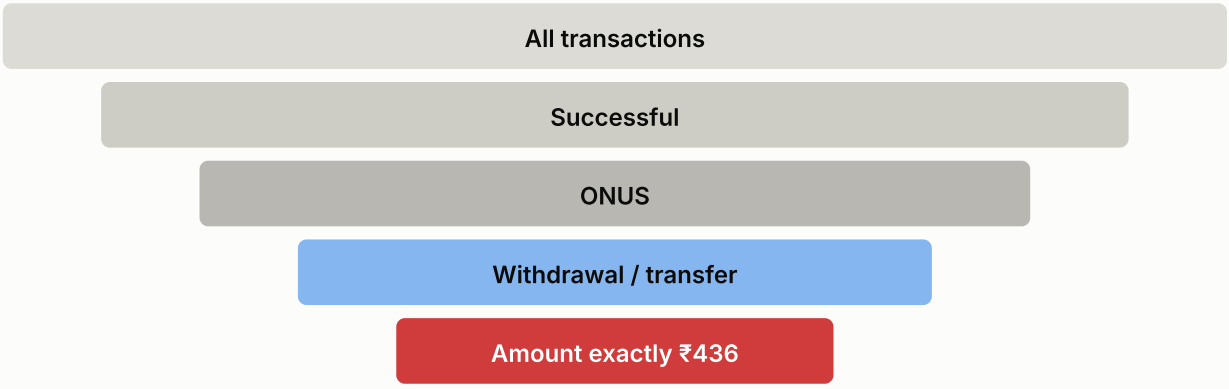
**When it fires.** Identify the destination account holder (KYC) and their link to the CSP; contact the source customers; recover the funds.

**Watch-outs.** "≥ 2 source accounts" is our addition. Without it, every customer who regularly pays the same family member was flagged. Set relative_min_sources = 1 for the literal rule.

06 YOUR RULE 6 Real-time · every transaction CSP analytics Low weight 0.3

# ₹436 transactions (PMJJBY premium)

Debits of exactly ₹436, the annual PMJJBY life-insurance premium, taken as a normal withdrawal or transfer.




## The fraud in practice

The PMJJBY premium should be auto-debited by the bank. When the same ₹436 shows up as a manual withdrawal or transfer at a CSP, the CSP may be collecting premiums in cash and not enrolling the customer, or selling the scheme without consent.

## How FraudLens detects it

1. Consider **successful, ONUS** transactions only.
2. Type must be a **withdrawal or transfer**.
3. Amount must be **exactly ₹436** (configurable).
4. Per CSP: count and amount of ₹436 debits; 3 or more raises a CSP flag.

## Worked example

| | | | | |
|---|---|---|---|---|
| 10:05 | Withdrawal | 3311… | ₹436 | FLAG |
| 10:30 | Withdrawal (OFFUS) | XXXX… | ₹436 | |
| 11:00 | Deposit | → 3311… | ₹436 | |

Only row 1 qualifies (OFFUS and deposits are excluded).

## Thresholds

| | |
|---|---|
| `pmjjby_amount` | **₹436** |
| `CSP flag` | **≥ 3 transactions** |

**WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)**


```
{"rule": "pmjjby_436", "severity": "low", "csp_id": "CSP0019", "txn_id": "TXN00163426", "amount": 436,
"message": "withdrawal ₹436 matches the PMJJBY premium amount", "account": "337096283114"}
```


### WHERE YOU SEE IT

- Verdict signal + alert `pmjjby_436`
- CSP column: ₹436
- CSP flag: “Repeated ₹436 debits” (+0.3)

### IN THE TEST RUN

Planted at CSP0019: **10 of 10 caught** (₹4,360). No other CSP had any.

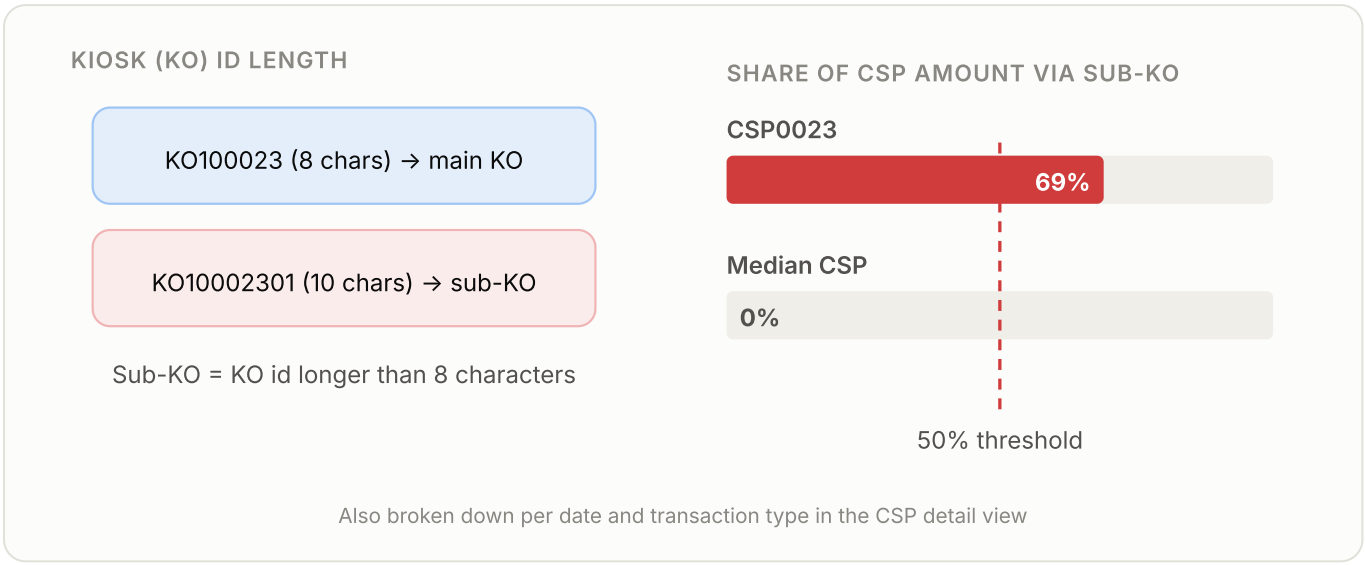
**When it fires.** Check whether each customer is actually enrolled in PMJJBY; if not, refund and treat it as mis-selling.

**Watch-outs.** The amount can be changed in config if the premium changes.

07 YOUR RULE 7 Score tag CSP analytics Info weight 0.05

# Sub-KO usage

Transcribe routed through sub-kiosks (sub-KOs) rather than the CSP's main kiosk, which can hide who really did the transaction.




Also broken down per date and transaction type in the CSP detail view

## The fraud in practice

A CSP can register sub-KOs operated by helpers. Heavy sub-KO use means most of the CSP's business is done by people outside direct oversight, a common route for unauthorised agents.

## How FraudLens detects it

1. A **sub-KO** is a KO id longer than 8 characters.
2. Each sub-KO transaction gets the **sub_ko** tag (small score, not stored as an alert).
3. Per CSP: **sub-KO share of count and of amount**, and a breakdown by **date and transaction type**.
4. A share above 50% (count or amount) raises a CSP flag.

## Worked example

| | | | | |
|---|---|---|---|---|
| 10:00 | Withdrawal | KO100023 | ₹2,000 | |
| 10:20 | Deposit | KO10002301 | ₹3,500 | SUB-KO |
| 11:10 | Withdrawal | KO10002301 | ₹1,200 | SUB-KO |

Rows 2–3 carry the tag. The CSP's daily breakdown shows the share by type.

## Thresholds

| | |
|---|---|
| sub_ko_min_length | **8 characters** |
| sub_ko_share_threshold | **50%** |

**WHAT AN INVESTIGATOR SEES · CSP REPORT (GET /API/V1/CSPS/CSP0023)**


```
{"csp_id": "CSP0023", "sub_ko_count": 1817, "sub_ko_count_share": 0.699, "sub_ko_amount_share": 0.69,
"risk_score": 59.2, "flags": ["High sub-KO share", "Part of a suspicious network", "Uses hub accounts"]}
```


**WHERE YOU SEE IT**

- Verdict tag `sub_ko` (weight 0.05)
- CSP column: Sub-KO amount share + detail table
- CSP flag: “High sub-KO share” (+0.2)

**IN THE TEST RUN**

Planted at CSP0023: **1,817 transactions via sub-KO = 70% of count, 69% of amount**. Median CSP: 0%.

**When it fires.** Confirm every sub-KO operator is registered and trained; visit the sub-KOs carrying most of the volume.

**Watch-outs.** If your kiosk ids have a different length convention, change sub_ko_min_length.

08 YOUR RULE 8 Score tag CSP analytics Info weight 0.05

# ₹50 transactions

Many tiny ₹50 transactions at one CSP, a sign of commission farming or account testing.




## The fraud in practice

Each transaction earns the CSP a fixed fee, so a stream of meaningless ₹50 transactions is cheap to create and profitable. Small amounts are also used to test stolen account details before larger fraud.

## How FraudLens detects it

1. Every transaction of exactly **₹50** (configurable) gets the **small_amount** tag.
2. Per CSP: count of ₹50 transactions.
3. A CSP flag is raised when there are **≥ 10** and they are **≥ 1%** of its transactions.

## Worked example

| | | | | |
|---|---|---|---|---|
| 09:30 | Withdrawal | 3401… | ₹50 | TAG |
| 09:41 | Withdrawal | 3122… | ₹50 | TAG |
| 09:55 | Withdrawal | 3367… | ₹50 | TAG |

Each row is tagged; the CSP total drives the flag.

## Thresholds

| | |
|---|---|
| `small_amount` | **₹50** |
| `CSP flag` | **≥ 10 and ≥ 1% of transactions** |

**WHAT AN INVESTIGATOR SEES · CSP REPORT (GET /API/V1/CSPS/CSP0029)**


```
{"csp_id": "CSP0029", "small_amount_count": 60, "count": 2510, "risk_score": 20, "flags": ["Many ₹50 transactions"]}
```


### WHERE YOU SEE IT

- Verdict tag `small_amount` (weight 0.05)
- CSP column: ₹50
- CSP flag: “Many ₹50 transactions” (+0.2)

### IN THE TEST RUN

Planted at CSP0029: **60 ₹50 transactions (2.4% of its volume)** → flagged. No other CSP reached 10.

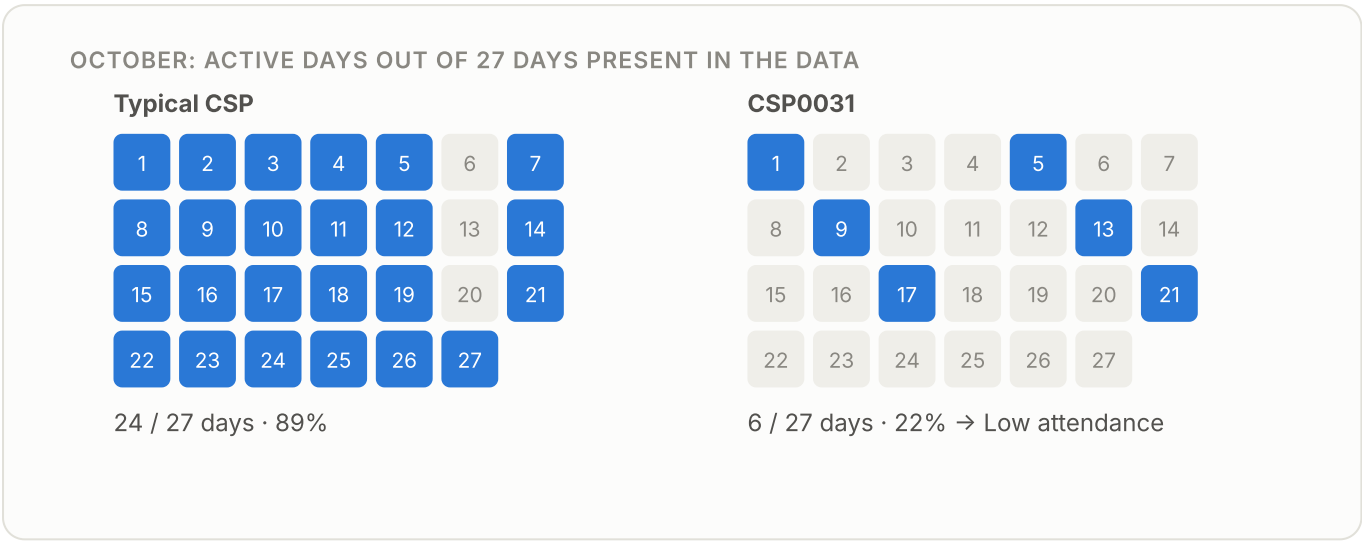
**When it fires.** Check whether the ₹50 transactions come from real, distinct customers; claw back commission on fabricated ones.

**Watch-outs.** Tags are deliberately not stored as alerts: they are too common to review one by one.

09 YOUR RULE 9 CSP analytics Medium CSP-level

# Attendance

A CSP that is only active on a few days a month: the outlet may not really be operating, or the login is being used remotely.




## The fraud in practice

A genuine CSP opens most working days. A CSP that appears for a handful of days, often with bursts of activity, may be a dormant outlet whose credentials someone else uses, or one that only switches on for fraudulent batches.

## How FraudLens detects it

1. For each CSP and calendar month, count **active days** (days with at least one successful transaction).
2. Divide by the **days present in the data** for that month, so partial months are fair.
3. Below **50%** in the latest month raises a flag; attendance also feeds peer anomaly (pattern 13).

## Worked example

| | | | | |
|---|---|---|---|---|
| Aug | 4 days in data | 1 active | 25% | |
| Sep | 30 days in data | 8 active | 27% | |
| Oct | 27 days in data | 6 active | 22% | LOW |

CSP0031 is flagged “Low attendance” for October.

## Thresholds

| | |
|---|---|
| `attendance_min_ratio` | **50%** |

**WHAT AN INVESTIGATOR SEES · CSP REPORT (GET /API/V1/CSPS/CSP0031)**


```
{"csp_id": "CSP0031", "active_days": 6, "attendance": 0.222, "risk_score": 60, "flags": ["Low attendance", "Peer anomaly high"]}
```


**WHERE YOU SEE IT**

- CSP column: Attendance (days · %)
- Monthly table in CSP detail
- CSP flag: “Low attendance” (+0.2)

**IN THE TEST RUN**

Planted CSP0031 (works ~1 day in 4): **6 of 27 days = 22%**. Median CSP: 89%.

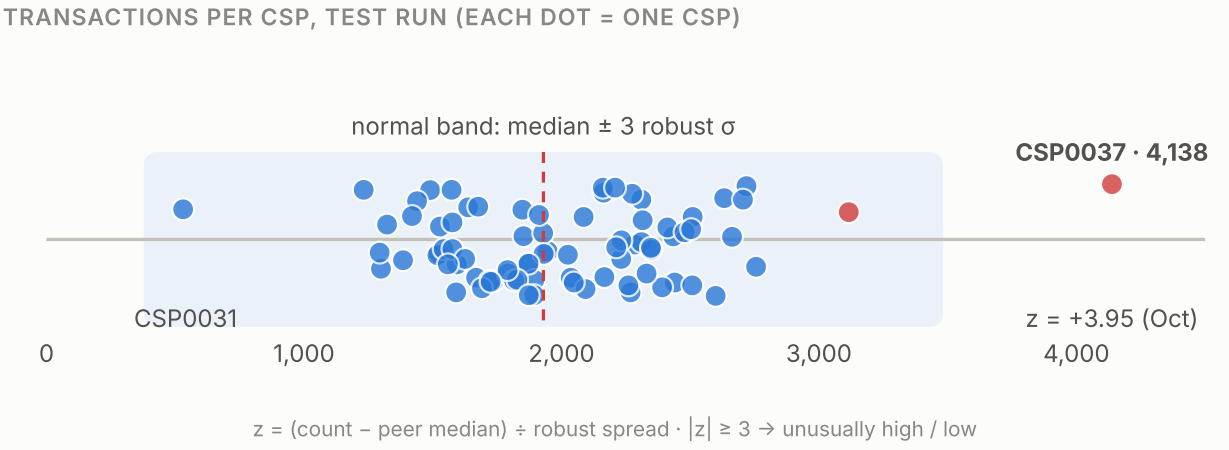
**When it fires.** Field visit to confirm the outlet operates; check the device and location used on the active days.

**Watch-outs.** Holidays affect everyone equally because the denominator is days present in the data, not calendar days.

10 YOUR RULE 10 CSP analytics Medium CSP-level

# Number of transactions

A CSP doing far more, or far fewer, transactions than comparable CSPs in the same month.




## The fraud in practice

Unusually high volume can mean commission farming, splitting or round-tripping. Unusually low volume can mean the outlet is barely operating. Either way it is worth a look.

## How FraudLens detects it

1. For each month, compare every CSP's transaction count with its **peer group** (the peer_group column, e.g. district; otherwise all CSPs).
2. Use a **robust z-score**: (count − peer median) ÷ robust spread (scaled MAD, never below 10% of the median).
3. |z| ≥ 3 marks the month **high** or **low**; the latest month drives the CSP flag.

## Worked example

| | | | | |
|---|---|---|---|---|
| Aug | 291 | z +2.8 | | |
| Sep | 2,037 | z +3.6 | high | |
| Oct | 1,810 | z +4.0 | high | FLAG |

CSP0037 is flagged “Unusual transaction count (high)”.

## Thresholds

| | |
|---|---|
| count_z_threshold | **3** |
| min_peer_group | **5 CSPs** |

**WHAT AN INVESTIGATOR SEES · CSP REPORT (GET /API/V1/CSPS/CSP0037)**


```
{"csp_id": "CSP0037", "count": 4138, "count_flag": "high", "risk_score": 66, "flags": ["Unusual transaction count (high)", "Peer anomaly high", "Uses hub accounts"]}
```


**WHERE YOU SEE IT**

- Monthly table in CSP detail (count vs peers z)
- CSP flag: “Unusual transaction count (high/low)” (+0.2)

**IN THE TEST RUN**

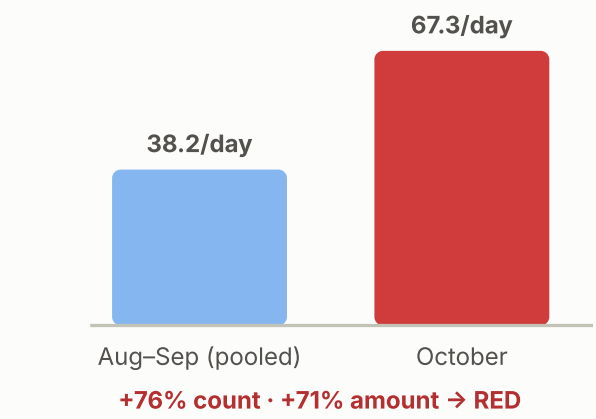
Planted CSP0037 (3× normal volume): **4,138 transactions vs median 1,930; z +4.0 in October**. CSP0041 (spike) also crossed z 3 in October.

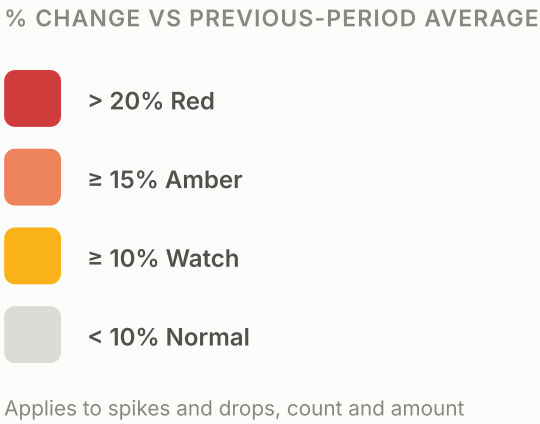
**When it fires.** Compare with footfall on a field visit; look at the CSP's split and round-trip shares for the same months.

**Watch-outs.** Robust statistics (median/MAD) stop one extreme CSP from distorting everyone else's baseline.

# Spike / drop

A sudden jump or fall in a CSP's own activity compared with its recent past.




## The fraud in practice

A CSP whose volume suddenly doubles may have started a fraud scheme or is being used by someone else; a sudden drop may mean it was warned off or that activity moved elsewhere.

## How FraudLens detects it

1. Compare **per-day rates**: this month's count (and amount) ÷ days in data, against the pooled rate of the **previous 3 months**.
2. Change = (current − previous) ÷ previous, for both count and amount; the larger one decides the level.
3. **> 20% red · ≥ 15% amber · ≥ 10% watch**. Applies to spikes and drops.
4. A latest-day vs 7-day-average figure is also shown in the CSP detail.

## Worked example

| | | | |
|---|---|---|---|
| Aug–Sep | 38.2 txns/day | | |
| October | 67.3 txns/day | +76% | RED |

CSP0041 is red for October: count +76%, amount +71%.

## Thresholds

| | |
|---|---|
| `spike_red` | **20%** |
| `spike_amber` | **15%** |
| `spike_watch` | **10%** |
| `spike_lookback_periods` | **3 months** |

**WHAT AN INVESTIGATOR SEES · CSP REPORT (GET /API/V1/CSPS/CSP0041)**


```
{"csp_id": "CSP0041", "count_change_pct": 76.141, "amount_change_pct": 71.005, "spike_level": "red",
"risk_score": 60.8, "flags": ["Unusual transaction count (high)", "Spike/drop red", "Peer anomaly medium"]}
```


### WHERE YOU SEE IT

- CSP column: Spike / drop
- Monthly table in CSP detail
- CSP flag: red +0.3 · amber +0.15

### IN THE TEST RUN

Planted CSP0041 (volume doubled for the last 3 weeks): **red, +76% count**. Across all CSPs: 4 red, 8 amber, 21 watch.

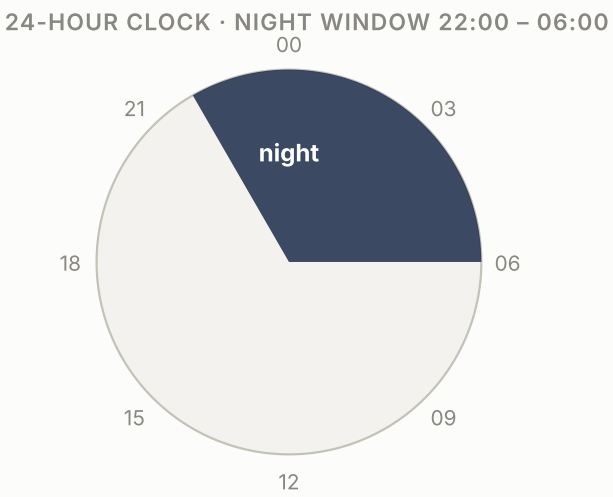
**When it fires.** Open the CSP's transactions for the month and find what drove the change: new accounts, one product, one time of day.

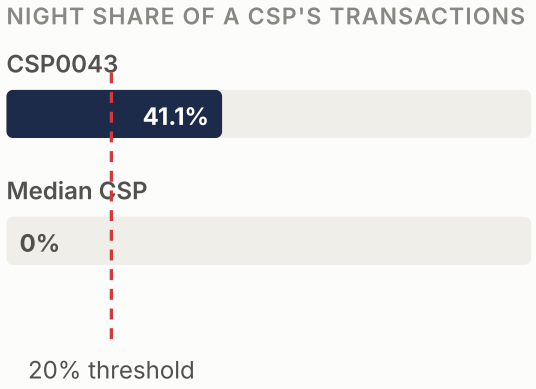
**Watch-outs.** Amounts vary naturally month to month, so expect some amber/watch levels on normal CSPs. Red is the actionable level.

12 YOUR RULE 12 Score tag CSP analytics Medium weight 0.1

# Night transactions

A CSP doing a large share of its business between 22:00 and 06:00, when an outlet should be closed.




## The fraud in practice

Real customers rarely visit a CSP at 2 a.m. A high night share suggests the login is being used remotely, or transactions are being fabricated after hours when nobody is watching.

## How FraudLens detects it

1. Every transaction between **22:00 and 06:00** (in IST) gets the **night** tag (weight 0.1).
2. Per CSP: night count and **night share** of all transactions.
3. A share **above 20%** raises a CSP flag.
4. High-value night transactions get their own alert (pattern 15).

## Worked example

| | | | | |
|---|---|---|---|---|
| 23:12 | Withdrawal | 3108… | ₹2,500 | TAG |
| 02:40 | Deposit | → 3376… | ₹4,000 | TAG |
| 11:15 | Withdrawal | 3290… | ₹1,000 | |

Night share for the CSP is recalculated on every refresh.

## Thresholds

| | |
|---|---|
| `night_start_hour` | **22** |
| `night_end_hour` | **6** |
| `night_share_threshold` | **20%** |

**WHAT AN INVESTIGATOR SEES · CSP REPORT (GET /API/V1/CSPS/CSP0043)**


```
{"csp_id": "CSP0043", "night_count": 949, "night_pct": 41.1, "risk_score": 40, "flags": ["Night transactions above 20%"]}
```


**WHERE YOU SEE IT**

- Verdict tag `night`
- CSP column: Night %
- CSP flag: “Night transactions above 20%” (+0.4)

**IN THE TEST RUN**

Planted CSP0043: **949 night transactions = 41.1%** of its volume. Median CSP: 0%.

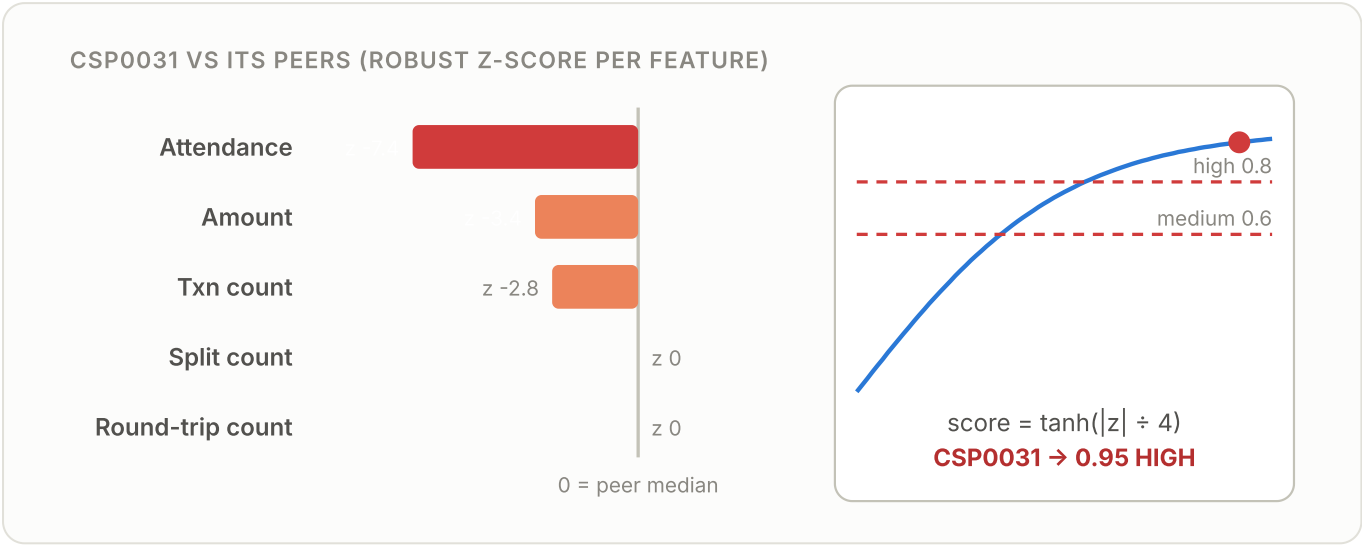
**When it fires.** Check the device and location of night logins; confirm opening hours; restrict night access if unexplained.

**Watch-outs.** Times without a zone are read as Asia/Kolkata; set timezone in config if your data is in another zone.

13 YOUR RULE 13 CSP analytics High CSP-level

# Anomaly vs peers

A CSP whose overall profile looks unlike the CSPs around it, even if no single rule fires.




## The fraud in practice

Fraud rarely shows up on one measure alone. A CSP with very low attendance, lower amounts and fewer transactions than every neighbour, or far more round trips, stands out once you compare it with its peers on several measures at once.

## How FraudLens detects it

1. Features per CSP: **transaction count, amount, attendance, split count, round-trip count**.
2. Peer group = the CSP's peer_group (e.g. district); groups smaller than 5 fall back to all CSPs.
3. Robust z-score per feature. Direction-aware: only **low** attendance and **high** split/round-trip count as risky; count and amount count both ways.
4. Feature score = tanh(|z| ÷ 4); CSP score = the maximum. **≥ 0.8 high, ≥ 0.6 medium**. The driving features are listed.

## Worked example

| | | | |
|---|---|---|---|
| Attendance | z −7.4 | 0.95 | HIGH |
| Amount | z −3.4 | 0.69 | MEDIUM |
| Txn count | z −2.8 | 0.60 | MEDIUM |

CSP0031 anomaly score 0.95 → “Peer anomaly high”.

## Thresholds

| | |
|---|---|
| anomaly_high | 0.8 |
| anomaly_medium | 0.6 |
| min_peer_group | 5 |

**WHAT AN INVESTIGATOR SEES · CSP REPORT (GET /API/V1/CSPS/CSP0031)**


```
{"csp_id": "CSP0031", "anomaly_score": 0.951, "anomaly_level": "high", "anomaly_drivers": {"amount": -3.365, "attendance": -7.368, "txn_count": -2.814}, "risk_score": 60, "flags": ["Low attendance", "Peer anomaly high"]}
```


### WHERE YOU SEE IT

- CSP column: Peer anomaly
- CSP detail: anomaly drivers with z-scores
- CSP flag: high +0.5 · medium +0.3

### IN THE TEST RUN

4 CSPs high, 1 medium. Drivers point at the planted cases: CSP0007 (round trips z +16), CSP0031 (attendance), CSP0037 (count/amount), CSP0003 (splits).

**When it fires.** Use the listed driver features to decide which pattern pages to investigate; treat it as a triage signal, not proof.

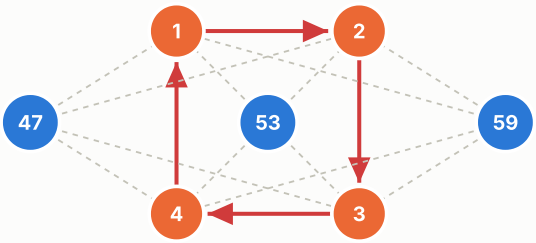
**Watch-outs.** Add a peer_group column (district / region) to compare like with like; urban and rural CSPs behave differently.

14 YOUR RULE 14 Real-time · every transaction CSP analytics Network High weight 0.45

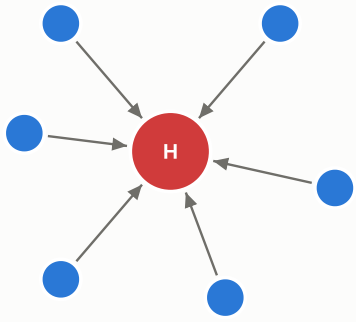
# Network / fraud rings

Groups of CSPs and accounts working together: money cycling between CSPs, hub accounts touched by many CSPs, and clusters linked by suspicious activity.

RING #1 FROM THE TEST RUN: CROSS-CSP MONEY CYCLE + HUB ACCOUNT




4 accounts · 16 transfers · ₹2,96,000 · 4 days · 3 CSPs




Hub account used at 8 different CSPs

## The fraud in practice

Organised fraud spreads across outlets so that no single CSP looks bad. Money goes round through accounts at different CSPs over several days, and a few mule accounts are paid from many CSPs.

## How FraudLens detects it

1. **Money cycles:** a global account graph across all CSPs and days; strongly connected components with ≥ 2 accounts are cycles, cross-CSP ones listed first.
2. **Hub accounts:** an account used at ≥ 5 different CSPs. A real-time **hub_account** alert fires when it reaches its 5th CSP.
3. **Rings:** CSPs and accounts joined by round trips, ₹1 tests, relative transfers, pass-through, threshold avoidance, hub links or cycle legs. Hub-only groups are not reported.
4. Each ring shows its CSPs, accounts, alerts by rule and an interactive graph.

## Worked example

| | | | | |
|---|---|---|---|---|
| Day 1 | CSP0047 | A1 → A2 | ₹20,000 | |
| Day 1 | CSP0053 | A2 → A3 | ₹19,000 | |
| Day 1 | CSP0059 | A3 → A4 | ₹18,000 | |
| Day 1 | CSP0047 | A4 → A1 | ₹17,000 | CYCLE |

Cycle across 3 CSPs: invisible to the per-CSP round-trip rule, caught here.

## Thresholds

| | |
|---|---|
| hub_min_csps | **5** |
| Cycle legs | **same filters as round trip** |

### WHAT AN INVESTIGATOR SEES · NETWORK (GET /API/V1/NETWORK) · FIRST CYCLE


```
{"id": 1, "accounts": ["323831234924", "353276595552", "389170945887", "390989207649"], "csps": ["CSP0047", "CSP0053", "CSP0059"], "txns": 16, "amount": 296000, "days": 4, "cross_csp": true}
```


### WHERE YOU SEE IT

- Network page: ring graph, cycles, hubs
- Alert `hub_account` (weight 0.45)
- CSP flags: “Part of a suspicious network” (+0.4), “Uses hub accounts” (+0.15)

### IN THE TEST RUN

15 cycles (1 cross-CSP: **4 accounts, 3 CSPs, 16 transfers, ₹2,96,000 over 4 days**), 20 hub accounts, 5 rings. Planted hub used at 8 CSPs is the top hub.

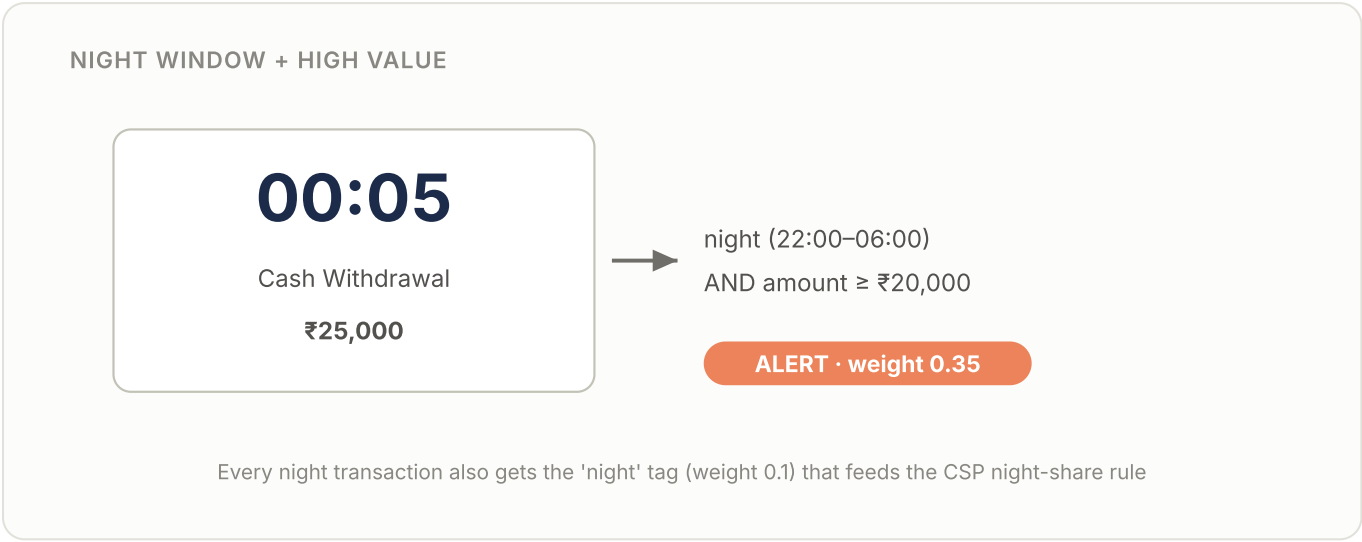
**When it fires.** Investigate the ring as one case: KYC every account, look for shared addresses and phone numbers, coordinate across the CSPs.

**Watch-outs.** Settlement / pool accounts look like hubs; list them in ignore_accounts.

15 ADDED BY FRAUDLENS Real-time · every transaction Medium weight 0.35

# High-value night transaction

A large transaction (₹20,000 or more) in the middle of the night.




## The fraud in practice

The night-share rule looks at CSP behaviour over time. This rule catches the single risky event straight away: a big cash-out at 00:05 deserves a second look before the money is gone.

## How FraudLens detects it

1. Transaction falls in the night window (22:00–06:00).
2. Amount is **≥ ₹20,000**.
3. Raises a stored alert immediately, on top of the night tag.

## Worked example

| | | | | |
|---|---|---|---|---|
| 00:05 | Withdrawal | 3221… | ₹25,000 | FLAG |
| 23:30 | Withdrawal | 3109… | ₹3,000 | TAG ONLY |

Row 1: **night_high_value** (0.35) + night tag (0.1) = score 0.415.

## Thresholds

| | |
|---|---|
| `night_high_value` | **₹20,000** |

**WHAT AN INVESTIGATOR SEES · SIGNAL IN THE WEBHOOK REPLY (TEST CASE)**


```
{"rule": "night_high_value", "severity": "medium", "weight": 0.35, "account": "3221…", "message": "withdrawal ₹25,000 at 00:05"}
```


**WHERE YOU SEE IT**

- Verdict signal + alert `night_high_value`
- CSP flag: +0.15

**IN THE TEST RUN**

Not present in the sample data (the generator plants no large night amounts). Verified by the automated test: ₹25,000 at midnight is flagged.

**When it fires.** Call the customer back before releasing funds; review device location and CCTV for the outlet.

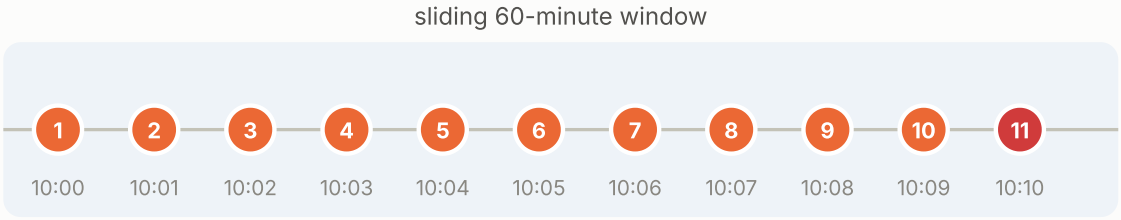
**Watch-outs.** Raise the threshold if your CSPs legitimately handle large late-evening deposits.

16 ADDED BY FRAUDLENS Real-time · every transaction Medium weight 0.45

# High velocity

An account sending out more than 10 transactions within an hour.

OUTGOING TRANSACTIONS FROM ONE ACCOUNT (TEST CASE)




**11 withdrawals in 11 minutes > 10 allowed per hour → the 11th is flagged**

Catches scripted / bulk draining of an account

## The fraud in practice

Someone who has taken over an account tries to drain it fast, often in many small pieces to stay under limits. Normal customers do not make a dozen withdrawals in an hour.

## How FraudLens detects it

1. For each account, keep the timestamps of recent **outgoing** transactions.
2. Count those in the **last 60 minutes**.
3. More than **10** → flag the transaction that crosses the line, and each one after.

## Worked example

| | | | | |
|---|---|---|---|---|
| 10:00–10:09 | 10 withdrawals | 3501… | ₹100+ | |
| 10:10 | Withdrawal | 3501… | ₹110 | FLAG |

11th withdrawal: **velocity** (0.45).

## Thresholds

| | |
|---|---|
| `velocity_window` | **1 hour** |
| `velocity_max_txns` | **10** |

**WHAT AN INVESTIGATOR SEES · SIGNAL IN THE WEBHOOK REPLY (TEST CASE)**


```
{"rule": "velocity", "severity": "medium", "weight": 0.45, "account": "3501…", "message": "11 outgoing transactions from 3501… within 1h0m0s", "details": {"count": 11, "window": "1h0m0s"}}
```


**WHERE YOU SEE IT**

- Verdict signal + alert `velocity`
- CSP flag: +0.2

**IN THE TEST RUN**

Not present in the sample data. Verified by the automated test: 11 withdrawals in 11 minutes is flagged.

**When it fires.** Temporarily stop further debits on the account and contact the account holder.

**Watch-outs.** Business accounts can be busy; exclude them with ignore_accounts or raise the limit.

17 ADDED BY FRAUDLENS Real-time · every transaction Network High weight 0.65

# Pass-through / mule account

Money that lands in an account and leaves again almost immediately: the signature of a money-mule account.

CSP0061 · MULE ACCOUNT 323615408763




out ÷ in within 30 min = 24,000 ÷ 25,000 = 96%

≥ 80% and credit ≥ ₹5,000 → pass-through

## The fraud in practice

Stolen or illicit money is moved through accounts belonging to 'mules', who withdraw it in cash quickly before it can be frozen. The account never holds the money for long.

## How FraudLens detects it

1. For each account, keep credits and debits from the last **30 minutes**.
2. On each debit, compare total out with total in over that window.
3. Flag if credits are **≥ ₹5,000** and debits are **≥ 80%** of them.

## Worked example

| | | | | |
|---|---|---|---|---|
| 13:00 | Transfer in | → M | ₹25,000 | |
| 13:12 | Withdrawal | M → | ₹24,000 | FLAG |

**pass_through** (0.65, high).

## Thresholds

| | |
|---|---|
| `pass_through_window` | **30 minutes** |
| `pass_through_ratio` | **80%** |
| `pass_through_min` | **₹5,000** |

### WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)


```
{"rule": "pass_through", "severity": "high", "csp_id": "CSP0061", "txn_id": "TXN00163525", "amount": 24000,
"message": "₹25,000 credited to 323615408763 and ₹24,000 moved out again within 30m0s", "account":
"323615408763"}
```


### WHERE YOU SEE IT

- Verdict signal + alert `pass_through`
- Links the CSP and account in rings
- CSP flag: +0.35

### IN THE TEST RUN

Planted at CSP0061: **5 of 5 caught** (₹25,000 in → ₹24,000 out in 12 minutes). Also fired on the cross-CSP cycle accounts.

**When it fires.** Hold any remaining funds; trace where the credit came from; consider a suspicious transaction report if it repeats.

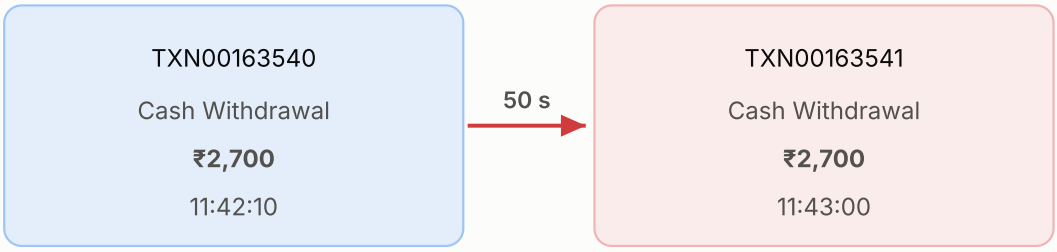
**Watch-outs.** ₹1 test deposits are left out of the credit total so they cannot trigger this rule.

18 ADDED BY FRAUDLENS Real-time · every transaction Medium weight 0.4

# Duplicate transaction

The same transaction posted twice within two minutes.

CSP0067 · SAME ACCOUNT, TYPE AND AMOUNT




**Same CSP + from + to + type + amount within 2 minutes → double posting or replay**

Re-sending the same txn_id is NOT a duplicate alert: the webhook is idempotent

## The fraud in practice

A double posting either charges the customer twice by mistake or is deliberately replayed by the CSP to take money twice.

## How FraudLens detects it

1. Key = CSP + from account + to account + type + amount.
2. If the previous transaction with the same key was **≤ 2 minutes** earlier → flag.
3. A repeated **txn_id** is different: the webhook treats it as a retry and returns the stored verdict.

## Worked example

| | | | | |
|---|---|---|---|---|
| 11:42:10 | Withdrawal | 3388… | ₹2,700 | |
| 11:43:00 | Withdrawal | 3388… | ₹2,700 | FLAG |

Second posting: **duplicate** (0.4).

## Thresholds

| | |
|---|---|
| `duplicate_window` | **2 minutes** |

**WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)**


```
{"rule": "duplicate", "severity": "medium", "csp_id": "CSP0067", "txn_id": "TXN00163541", "amount": 2700,
"message": "Same withdrawal of ₹2,700 repeated 50s after TXN00163540", "account": "391784596812",
"related_txns": ["TXN00163540", "TXN00163541"]}
```


**WHERE YOU SEE IT**

- Verdict signal + alert `duplicate`
- CSP flag: +0.15

**IN THE TEST RUN**

Planted at CSP0067: **8 of 8 caught** (plus 1 natural repeat elsewhere).

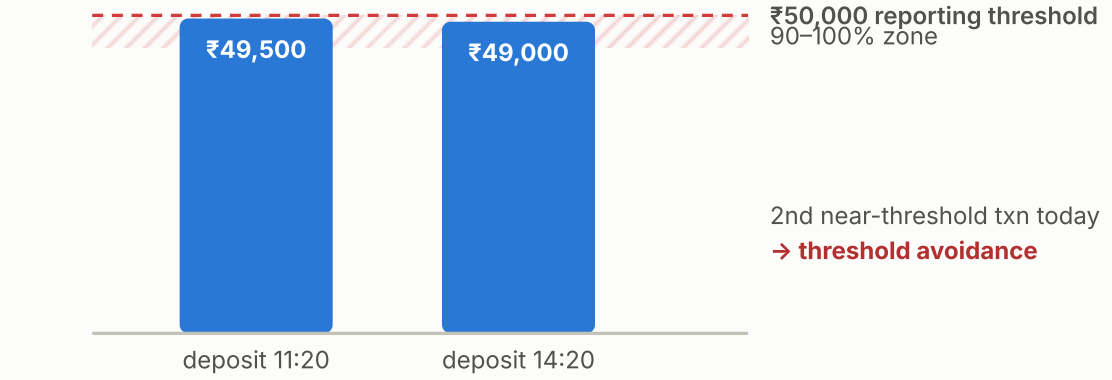
**When it fires.** Check whether the customer was debited twice and reverse the duplicate; look for repeated replays by the same CSP.

**Watch-outs.** Two genuinely identical withdrawals within 2 minutes are rare but possible.

19 ADDED BY FRAUDLENS Real-time · every transaction Network Medium weight 0.45

# Threshold avoidance (structuring)

Repeated amounts just below the ₹50,000 reporting threshold on the same account and day.




## The fraud in practice

Cash transactions of ₹50,000 and above need PAN details and are reported. Depositing ₹49,500 and then ₹49,000 the same day avoids that while still moving ~₹1 lakh.

## How FraudLens detects it

1. Zone = **90–100%** of the reporting threshold (₹45,000 – ₹49,999).
2. Count zone transactions per account per day.
3. The **2nd** and later ones are flagged.

## Worked example

| | | | | |
|---|---|---|---|---|
| 11:20 | Deposit | → 3484… | ₹49,500 | |
| 14:20 | Deposit | → 3484… | ₹49,000 | FLAG |

Second deposit: **threshold_avoidance** (0.45).

## Thresholds

| | |
|---|---|
| `reporting_threshold` | **₹50,000** |
| `near_threshold_ratio` | **90%** |

**WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)**


```
{"rule": "threshold_avoidance", "severity": "medium", "csp_id": "CSP0073", "txn_id": "TXN00163563", "amount": 49000, "message": "2 transactions today on 348488963493 just below the ₹50,000 threshold", "account": "348488963493"}
```


### WHERE YOU SEE IT

- Verdict signal + alert `threshold_avoidance`
- CSP flag: +0.3

### IN THE TEST RUN

Planted at CSP0073: **4 of 4 caught**. CSP0073 risk 85 (high).

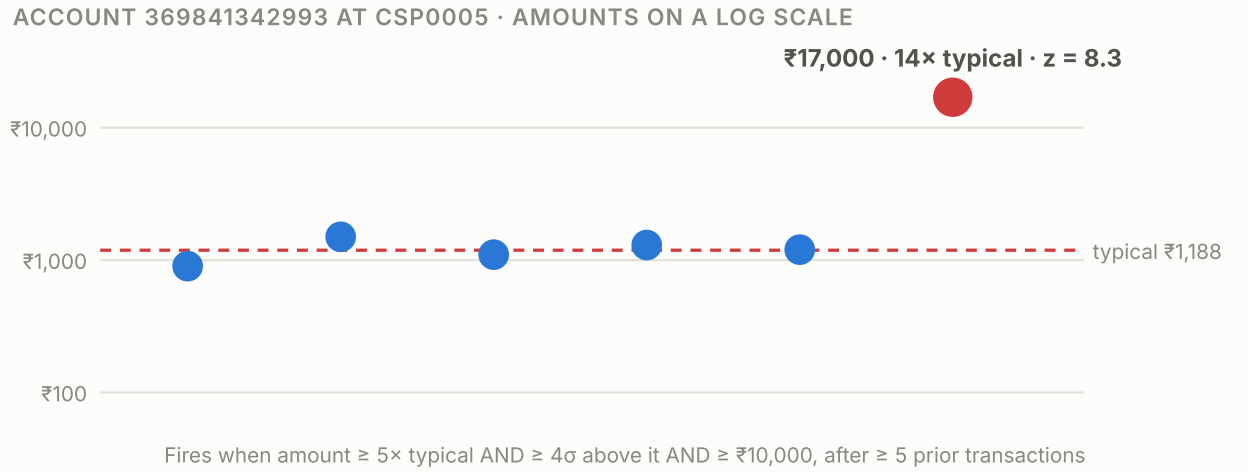
**When it fires.** Ask for PAN and source of funds; review the account's other days; consider regulatory reporting.

**Watch-outs.** Complements the split rule, which works on the ₹30,000 unit rather than the reporting limit.

20 ADDED BY FRAUDLENS Real-time · every transaction Medium weight 0.4

# Amount outlier

A transaction far larger than anything the account normally does.




## The fraud in practice

An account that usually moves around ₹1,000 suddenly moves ₹17,000. That is typical of account takeover or of an account being lent out.

## How FraudLens detects it

1. Keep a running mean and spread of **log(amount)** per account (money is heavy-tailed, so a log scale is fairer).
2. After **≥ 5** prior transactions, flag if amount is **≥ ₹10,000**, **≥ 5× typical** and **≥ 4σ** above it.

## Worked example

| | | | | |
|---|---|---|---|---|
| Earlier | 5 transactions | 3698… | ~₹1,200 | |
| Today | Withdrawal | 3698… | ₹17,000 | FLAG |

**amount_outlier** (0.4), z = 8.3.

## Thresholds

| | |
|---|---|
| `outlier_min_history` | **5** |
| `outlier_z` | **4** |
| `outlier_min_amount` | **₹10,000** |

**WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)**


```
{"rule": "amount_outlier", "severity": "medium", "csp_id": "CSP0005", "txn_id": "TXN00009406", "amount": 17000,
"message": "₹17,000 is far above the usual ₹1188 for 369841342993 (5 previous txns)", "account":
"369841342993"}
```


**WHERE YOU SEE IT**

- Verdict signal + alert `amount_outlier`

**IN THE TEST RUN**

12 alerts in the sample, e.g. CSP0005: ₹17,000 against a typical ₹1,188.

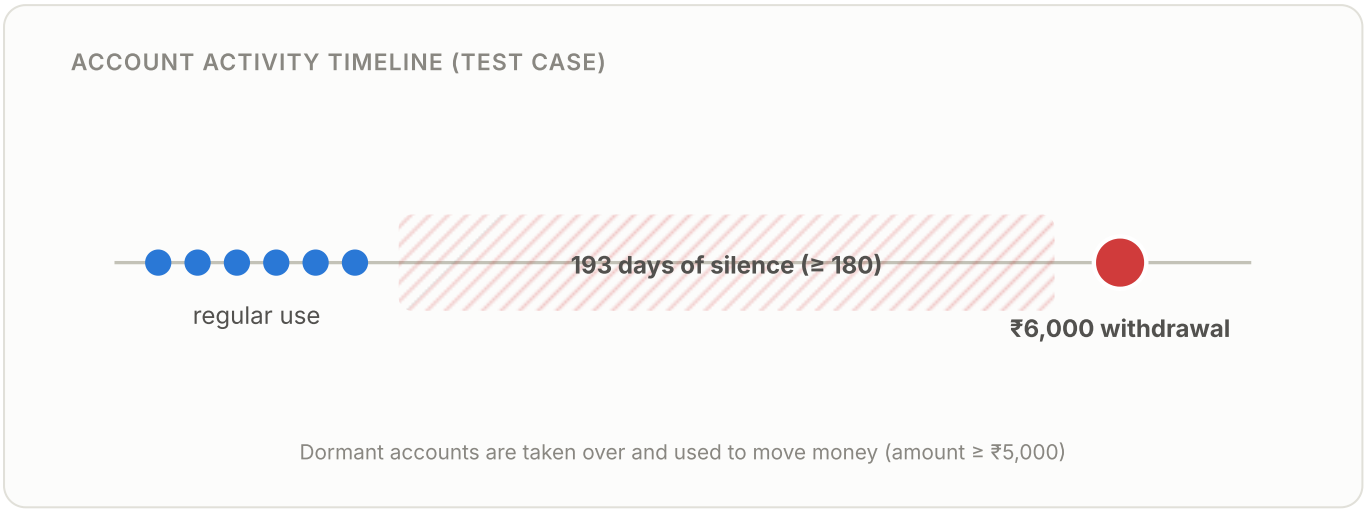
**When it fires.** Call the account holder to confirm the transaction before or soon after it completes.

**Watch-outs.** New accounts (fewer than 5 transactions) are not scored.

21 ADDED BY FRAUDLENS Real-time · every transaction Medium weight 0.45

# Dormant account reactivated

An account silent for 180+ days that suddenly moves ₹5,000 or more.




## The fraud in practice

Dormant accounts belong to people who are not watching them, which makes them attractive to take over and use for laundering.

## How FraudLens detects it

1. Track each account's last-seen time.
2. If the gap before this transaction is **≥ 180 days** and amount is **≥ ₹5,000** → flag.

## Worked example

| | | | | |
|---|---|---|---|---|
| Jan–Apr | 6 transactions | 4410… | ₹500–550 | |
| Oct (193 days later) | Withdrawal | 4410… | ₹6,000 | FLAG |

**dormant_reactivation** (0.45).

## Thresholds

| | |
|---|---|
| `dormant_after` | **180 days** |
| `dormant_min_amount` | **₹5,000** |

**WHAT AN INVESTIGATOR SEES · SIGNAL IN THE WEBHOOK REPLY (TEST CASE)**


```
{"rule": "dormant_reactivation", "severity": "medium", "weight": 0.45, "account": "4410…", "message": "4410… was inactive for 193 days before this ₹6,000", "details": {"inactive_days": 193}}
```


**WHERE YOU SEE IT**

- Verdict signal + alert `dormant_reactivation`
- CSP flag: +0.15

**IN THE TEST RUN**

Not present in the 60-day sample (needs a longer history). Verified by the automated test.

**When it fires.** Verify the customer's identity in person (biometric / KYC) before allowing further debits.

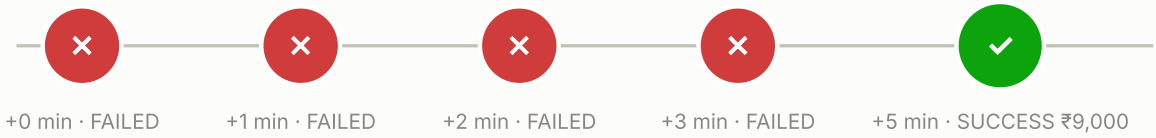
**Watch-outs.** The rule needs history in the cache: use retention of at least 180 days for it to work.

22 ADDED BY FRAUDLENS Real-time · every transaction High weight 0.55

# Success after failed attempts

Several failed attempts on an account followed by a success: someone guessing until it worked.

CSP0071 · ONE ACCOUNT WITHIN 10 MINUTES



+0 min · FAILED | +1 min · FAILED | +2 min · FAILED | +3 min · FAILED | +5 min · SUCCESS ₹9,000

**≥ 3 failures then a success → PIN / credential guessing that finally worked**

## The fraud in practice

Repeated failures (wrong PIN, wrong biometric, insufficient checks) followed by a success often mean the person at the counter is not the account holder.

## How FraudLens detects it

1. Record failed transactions per account.
2. On the next success, count failures in the previous **10 minutes**.
3. **≥ 3** → flag, then reset.

## Worked example

| | | | | |
|---|---|---|---|---|
| +0 to +3 min | 4 × Withdrawal | 3013… | ₹9,000 | FAILED |
| +5 min | Withdrawal | 3013… | ₹9,000 | FLAG |

**failed_burst** (0.55, high).

## Thresholds

| | |
|---|---|
| `failed_burst_count` | **3** |
| `failed_burst_window` | **10 minutes** |

**WHAT AN INVESTIGATOR SEES · ALERT (GET /API/V1/ALERTS)**


```
{"rule": "failed_burst", "severity": "high", "csp_id": "CSP0071", "txn_id": "TXN00163556", "amount": 9000,
"message": "Succeeded after 4 failed attempts on 301314498258 within 10m0s", "account": "301314498258"}
```


**WHERE YOU SEE IT**

- Verdict signal + alert `failed_burst`
- CSP flag: +0.25

**IN THE TEST RUN**

Planted at CSP0071: **4 of 4 caught**.

**When it fires.** Contact the account holder; check whether the person at the counter is the owner; reset credentials if needed.

**Watch-outs.** Needs the status column; rows without status are treated as successful.

A

# Appendix A · Input format and API

## Columns

| FIELD | NOTES |
|---|---|
| **amount** * | 5050 · “₹5,050.00” · “Rs. 5050” |
| **csp_id** * | also “CSP”, “Agent ID”, “BC ID”... |
| txn_id | used for safe retries (idempotency) |
| timestamp, or date + time | ISO, dd-mm-yyyy, dd/mm/yyyy hh:mm, 02-Jan-2026, Excel dates; day-first; IST if no zone |
| txn_type | free text → deposit / withdrawal / transfer / loan / initial deposit |
| from_account, to_account | or one “account” column; masked XXXXXX recognised |
| ko_id | needed for sub-KO |
| customer_charge | needed for the Money Transfer → Deposit split rule |
| channel | ONUS / OFFUS (empty = ONUS) |
| status | success / failed (empty = success) |
| peer_group | district / region for peer comparison |

* required. Header names are matched ignoring case, spaces and punctuation.

## Endpoints

| | |
|---|---|
| POST /api/v1/transactions | webhook: one JSON → one verdict; array / CSV → batch |
| POST /api/v1/upload | sheet upload (multipart “file”) |
| GET /api/v1/transactions/{id} | one transaction with signals |
| GET /api/v1/alerts | filter: rule, csp, account, severity |
| GET /api/v1/csps[/{id}] | CSP risk reports |
| GET /api/v1/network | cycles, hubs, rings |
| GET /api/v1/stream | live alerts (Server-Sent Events) |
| GET /api/v1/export/*.csv | alerts.csv, csps.csv |

## Webhook call

```
POST /api/v1/transactions
{"txn_id":"T1001","timestamp":"2026-10-27 14:05:00",
 "csp_id":"CSP0042","from_account":"300012345678",
 "txn_type":"Cash Withdrawal","amount":5050}
```

```
{"txn_id":"T1001","risk_score":0.4,"risk_level":"low",
 "action":"ALLOW","latency_us":7.9,
 "signals":[{"rule":"extra_charge","weight":0.4,
   "message":"withdrawal/transfer is ₹5,050 on a round
   ₹5,000: likely ₹50 extra taken from the
customer"}]}
```

Set FRAUD_API_KEY to require an X-API-Key header on every API call.

B

# Appendix B · Configuration

Every threshold lives in one JSON file. Copy `config.example.json`, change what you need, and start with `-config my.json`. Amounts are in rupees; durations like “30m”, “1h”.

| | |
|---|---|
| timezone | "Asia/Kolkata" |
| ignore_accounts | [] |
| split_unit | 30000 |
| round_trip_min_amount | 100 |
| round_trip_max_depth | 8 |
| extra_charge_min_base | 100 |
| test_deposit_amount | 1 |
| relative_max_csps | 3 |
| relative_min_txns | 3 |
| relative_min_sources | 2 |
| pmjjby_amount | 436 |
| extra_charge_share_pct | 0.5 |
| split_share_pct | 5 |
| sub_ko_min_length | 8 |
| sub_ko_share_threshold | 0.5 |
| small_amount | 50 |
| attendance_min_ratio | 0.5 |
| count_z_threshold | 3 |
| spike_red | 0.2 |
| spike_amber | 0.15 |
| spike_watch | 0.1 |
| spike_lookback_periods | 3 |
| night_start_hour | 22 |
| night_end_hour | 6 |
| night_share_threshold | 0.2 |
| night_high_value | 20000 |
| anomaly_high | 0.8 |
| anomaly_medium | 0.6 |
| min_peer_group | 5 |
| hub_min_csps | 5 |
| velocity_window | "1h0m0s" |
| velocity_max_txns | 10 |
| pass_through_window | "30m0s" |
| pass_through_ratio | 0.8 |
| pass_through_min | 5000 |
| duplicate_window | "2m0s" |
| reporting_threshold | 50000 |
| near_threshold_ratio | 0.9 |
| outlier_min_history | 5 |
| outlier_z | 4 |
| outlier_min_amount | 10000 |
| dormant_after | "4320h0m0s" |
| dormant_min_amount | 5000 |
| failed_burst_count | 3 |
| failed_burst_window | "10m0s" |
| review_score | 0.5 |
| block_score | 0.8 |

**weights** (optional) overrides any signal weight, e.g. `{"weights": {"extra_charge": 0.5}}`. **ignore_accounts** lists settlement or pool accounts to leave out of account-level and graph rules.

C

# Appendix C · Testing, performance and next steps

### Detection on planted cases

| PATTERN (PLANTED CSP) | PLANTED | CAUGHT |
|---|---|---|
| Round trip (CSP0007) | 15 loops | 15 |
| ₹1 test deposit (CSP0013) | 6 | 6 |
| Relative account (CSP0017) | 1 account | 1 |
| ₹436 (CSP0019) | 10 | 10 |
| Pass-through (CSP0061) | 5 | 5 |
| Duplicate (CSP0067) | 8 | 8 |
| Failed burst (CSP0071) | 4 | 4 |
| Threshold avoidance (CSP0073) | 4 | 4 |
| Cross-CSP cycle (CSP0047/53/59) | 1 | 1 |
| CSP-level: split, extra, sub-KO, ₹50, attendance, count, spike, night | 8 CSPs | 8 flagged |

Plus automated tests for every rule (including velocity, dormant and high-value night, which the sample does not contain), the file parsers, the API and restart recovery — all passing, race detector clean.

### Performance (4-vCPU container)

| | |
|---|---|
| Decision time per transaction (median / p99) | 5.1 µs / 107 µs |
| Webhook round trip incl. HTTP | ≈ 0.3 ms |
| Upload + analyse 1,63,569 rows | ≈ 3.3–3.9 s |
| Restart: rebuild cache from log | ≈ 2.9 s |
| Memory per cached transaction | ≈ 0.5–1 KB |

### Before going live

- **Tune on real data.** All results here are on synthetic data; run a month of real transactions and adjust thresholds per region.
- **List settlement / pool accounts** in ignore_accounts.
- **Add peer_group** (district) for fairer peer comparison.
- **Set retention** (e.g. 180+ days, so the dormant rule works) to bound memory.
- **Set an API key** and run behind HTTPS.